



ALWAYS ON

Field	Value
Document	Complete single-file architecture, integration, security, operations, evidence, and active-work report
License	CC BY-NC-SA
Project origin	Building ~2010 · Drone ~2012 · Linux systems ~2023
Website	https://www.300x3.com
Supporting plan	https://archive.org/details/@scott_widmann
Created with	Bluebeam and LibreDraw (PDF project plan), Perplexity.ai, Cline.bot
Revision	2026-09-28 (v2 — restructured; see Appendix A)
Current main host	One Kubuntu desktop
Peripheral host	One Raspberry Pi 5 and Autopilot Module running KaliOS and ArduPilot
Format rule	Tables and topology diagrams are primary; original detailed commands/evidence are retained in-place below for operational completeness
Status	Current, Next, Later, Waiting for, Issue
Reading order	ES.1-ES.3 are the current-state summary; sections 1-21 are the authoritative detailed record. Where they disagree, ES.1 governs.

Authority rule. ES.1 (Current Architecture Corrections) is the single source of truth for current architecture. Any statement elsewhere in this document that conflicts with ES.1 is **superseded**. Deviations are recorded in section 18 with rationale and compensating controls; a recorded deviation never silently overrides ES.1 — an open deviation means ES.1 describes the target and section 18 describes what is actually deployed.

Contents

- **ES** Executive Summary — ES.1 Current Architecture Corrections · ES.2 Master Topology · ES.3 Current Work · ES.4 Detailed System Record
- **1** System Purpose · **2** Platform Baseline · **3** High-Level Architecture · **4** Security, Isolation, and Data Policy
- **5** Network Domains and Controlled External Access · **6** Component Boundaries and Operator Access
- **7** Public Storefront and Payment Policy · **8** Mapping and Photogrammetry · **9** Field and LoRa Architecture
- **10** Simulation Architecture · **11** Ledger, Provenance, Archive, and IPFS
- **12** Host Installation and Configuration · **13** Podman Runtime and Quadlet Policy
- **14** Secrets, Service Identity, and Version Controls · **15** Sales, Mastodon, OpenClaw, and Local AI
- **16** Scripts and Operational Standards · **17** Backup, Restore, and Monitoring
- **18** Approved Deviations and Open Decisions · **19** Work Queue and Issue Log
- **20** Runtime Placement and Verification Evidence · **21** Status References · **Appendix A** Revision History

Executive Summary

ES.1 Current Architecture Corrections

Subject	Current plan; replaces any contrary older text below
OpenClaw	Public marketing/contact across website chat, email, Mastodon, approved social channels; direct Mastodon publisher; local output is standardized PDFs for order/follow-up/support/payment workflows
LM Studio	Fundamental local LLM host for OpenClaw
Browser	Konqueror; Tokodon removed completely
RNode client	MeshChatX exclusively
Radios	PEOPLE-RADIO = public human chat; DRONE-RADIO = authenticated private drone mission/status traffic
QGroundControl	Desktop primary mission planning; KaliOS RPi5 fallback/out-of-range mission-update operation
Prometheus	Independent security monitoring, TSDB, rule evaluation, alerting, and evidence layer; works without Grafana
Grafana	Optional protected visualization only
Corda	Blockchain-enabled accounting, ledger, receipt, entitlement, fulfillment, provenance, and approved state-transition system
Corda persistence	PostgreSQL. Corda was scaffolded on H2 by mistake; PostgreSQL is the correct database. The cordadb placement on the host PostgreSQL 18 cluster is a separate logical database with its own roles and backup scope, and is correct
IPFS	Post-sale marking/transfer only for approved map/imagery and telemetry/product-operational packages; no Corda/ledger/accounting/archive dependency
Backup/archive	Local authority + encrypted restic + encrypted pCloud replication; independent of IPFS and Corda
Secret authority	KDE Wallet; services needing Wallet secrets start after KDE login
GPU policy	Priority: desktop/Konqueror → active LM Studio/OpenClaw → SketchUp/SketchUp Web → ROS/Gazebo/SITL → WebODM batch
Additive fabrication	MainsailOS/Mainsail/Moonraker/Klipper on printer-local BigTreeTech CB1/RPi; no OrcaSlicer reference
CNC	No bCNC or current CNC software claim
Home automation	Domoticz/RPi for HVAC, doors/locks/access, security/alarms, lighting/scenes, environmental sensing; separate from printers and future building robotics
Completed work	Incorporated as Current; not listed as completed work items
Work 000600	Confirm local Mastodon operation
Work 000601	Low-priority confirm Metabase persistence and first protected read-only reporting query
Work 000700	Low-priority MeshChatX/RNode minor RF-interference characterization

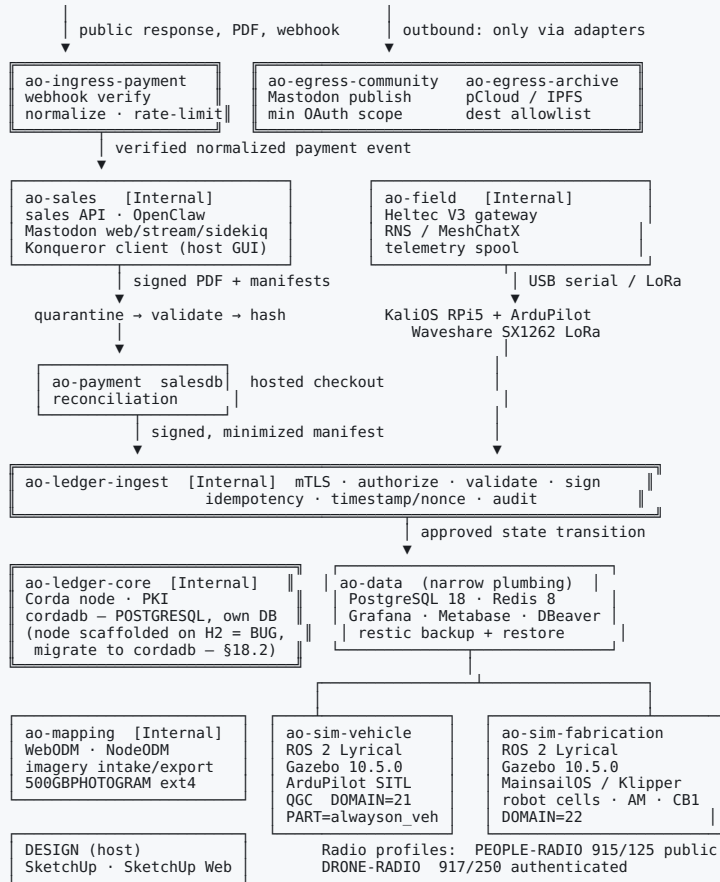
ES.2 Master Topology

This is the **single authoritative diagram of the entire project**. Every other diagram in this document is a detail view of one part of it and must not contradict it. Domain names, network names, and paths are abbreviated here and defined in full in the sections referenced.

ALWAYS ON – MASTER TOPOLOGY
Single source of truth. Later sections are detail views of this diagram.

EXTERNAL (no route to any internal service below except via an adapter)

storefront 300x3.com – static pCloud Public Folder: products, docs, legal
Mastodon mastodon.300x3.com – federated via dedicated Cloudflare Tunnel
payment provider (PayPal / Zelle / stablecoin) – hosted checkout only
Mastodon.social and other remote instances; pCloud archive (encrypted egress)



POST-SALE TRANSFER (authorized recipient only – no Corda/archive dependency)

approved map/imagery or telemetry package → package · hash · recipient
authorization → IPFS transfer (private swarm / pinning / encryption)

SECRETS – KDE Wallet is the authority (ES.1)

Plasma login → kwalletd6 unlock → fetch-kwallet-secret.sh (ExecStartPre, ≤60s)
→ 0600 env file per unit. Services needing Wallet secrets start after login
BY DESIGN. All services run under the operator account; no separate service user.

MONITORING – Prometheus is independent; Grafana is optional

host + services + exporters → Prometheus TSDB · rules · alerts → works alone

→ Grafana (optional viz only)
ao-admin carries Prometheus, node_exporter, Grafana, Metabase, backup/restore

Reading the diagram.

Question	Answer
What is the only public entry?	The storefront, the federation hostname, and the adapters (ao-ingress-payment, ao-egress-community, ao-egress-archive, ao-build-update). Nothing else reaches an internal service.
Where does money move?	Hosted checkout at the provider → ao-ingress-payment → verified event → ao-payment / salesdb → signed manifest → ao-ledger-ingest → Corda. Cards are never handled locally.
Where is the ledger truth?	ao-ledger-core on cordadb in PostgreSQL 18, a separate database with its own roles and backup scope. PostgreSQL is authoritative; the H2 scaffold is the defect (\$18.2).
What can the internet never reach?	Every ao-* workload network. All are Internal=true with no public listener.
What crosses a domain boundary?	Only a signed, minimized manifest through ao-ledger-ingest, under mTLS with authorization, replay defence, idempotency, and audit.
Where do secrets come from?	KDE Wallet, after Plasma login, by design. See §14.1.1.

Question	Answer
Is monitoring possible without Grafana?	Yes. Prometheus is the collector, rule engine, and alerting layer; Grafana is optional visualization.

ES.3 Current Work

ID	Area	Next action	Acceptance criteria	Priority
000600	Mastodon	Confirm authoritative service account, loopback origins, Cloudflare route, egress isolation, and OpenClaw publisher	Health/WebFinger/actor/origin/egress/API publisher/audit-PDF path pass	Normal
000601	Metabase	Confirm persistence and first protected read-only reporting path	State survives restart; read-only role/view query succeeds; no source writes	Low
000700	MeshChatX/RNode	Observe and characterize the observed minor RF interference	Evidence recorded either way: if the interference is measurable, capture RSSI/SNR/noise/loss/retry/airtime/cross-band data; if it is benign ambient noise, record that finding. No corrective action is required unless measurement shows a real fault. Closure = a recorded characterization, not a fix	Low
—	Prometheus	Deploy TSDB/exporters/rules/alerts	Alert tests pass without Grafana	Normal
—	Corda	Complete key/certificate and synthetic ledger test	Confirmed state in dedicated Corda DB/projection	Normal
—	WebODM folders	Create/validate tree, ownership, sentinel, checks	WebODM starts only with required validated storage	Normal
—	Payment adapters	Build provider/reconciliation evidence path	Valid evidence produces normalized state and eligible ledger manifest	Normal
—	Wallet/Quadlet	Test login/restart/re-login delivery	Services receive named secrets only after KDE login	Normal
—	GPU	Test scheduling/admission	LM Studio/SketchUp/Gazebo/WebODM match policy	Normal

ES.4 Detailed System Record

The remaining tables, commands, evidence, paths, full interface inventory, operational requirements, payment/ledger forms, Mastodon details, installation procedures, backup rules, and service records follow, numbered 1–21. Where an older statement conflicts with ES.1, ES.1 governs.

Architecture, Installation, Configuration, Operations, and Status

Operator warning. Everything here is difficult until it is easy. These tools are sharp and dangerous, and if you disrespect them they will can/will harm you — the same as any trip, fall, or car ride. Perhaps not as bad, perhaps worse. In my opinion, ignoring them is more dangerous than understanding them.

Document Status and Reading Guide

This README is the single authoritative document for the ALWAYS ON project. It contains the intended architecture, implemented configuration, operational requirements, validation evidence, approved deviations, known issues, and work queue.

Every material statement in this document belongs to one of the following categories:

Category	Meaning
Architecture requirement	Mandatory final-state design constraint
Implemented	Verified as deployed or tested on the current host
Planned	Approved design not yet implemented
Blocked	Requires an operator decision, credential, key ceremony, hardware connection, or other prerequisite
Deviation	Approved difference between intended architecture and current implementation
Work item	A discrete task with acceptance criteria

Category	Meaning
Issue	A recorded defect, ambiguity, or implementation risk

When the current implementation differs from an architecture requirement, the difference must be recorded in **Approved Deviations and Open Decisions** with a rationale, compensating controls, owner, and resolution condition.

Last consolidated review: 2026-08-31

1. System Purpose

ALWAYS ON is a compartmentalized, on-premises platform supporting an automated approximately 160-square-foot modular live/fabricate facility and an accompanying modular micro-aircraft carrier. Both of which grow to generally any size / quantity.

The platform supports:

- Drone telemetry and field communications.
- Photogrammetry and mapping.
- Vehicle simulation.
- Fabrication, facility, inventory, kitchen, and logistics simulation.
- Static public sales content.
- Hosted payment checkout and receipt generation.
- Mastodon-based customer and community follow-up.
- Local LM Studio and OpenClaw-assisted support workflows.
- Corda-backed provenance, receipts, entitlements, and approved state records.
- Encrypted pCloud archival replication and controlled IPFS artifact distribution.

The system is designed around **strict isolation**. Sales, AI, payments, mapping, field telemetry, vehicle simulation, fabrication simulation, archive, and ledger services must not share broad networks, credentials, writable storage, databases, or unrestricted host access.

The current workstation is a development, integration, and validation host. It uses Kubuntu 26.04 LTS software, an AMD CPU, and an EVGA NVIDIA GTX 1080. Future compute-intensive production workloads may move to an immersion-cooled server rack and a Raspberry Pi edge-computing cluster.

Kubuntu is appropriate for the current workstation role because KDE supports QGroundControl, Gazebo visualization, GPU diagnostics, Konqueror, and general engineering workflows while retaining an Ubuntu LTS package base. Ubuntu 26.04 LTS standard support is scheduled through April 2031.

Podman is the only supported container runtime. Containers are managed through systemd Quadlet definitions rather than Docker Compose, shell-wrapper orchestration, or a Docker daemon.

2. Platform Baseline

2.1 Intended Platform Standard

Area	Architecture requirement
Host OS	Kubuntu 26.04 LTS workstation
Current CPU/GPU	AMD CPU and EVGA NVIDIA GTX 1080
Future compute	Immersion-cooled server rack and Raspberry Pi edge cluster
Container engine	Podman only
Container lifecycle	systemd and Podman Quadlet
Public website	Static HTML in pCloud Public Folder
Payments	Provider-hosted checkout and verified payment events; no local card handling; local stablecoin processing
Sales and support	Sales API, PostgreSQL, Mastodon integration, OpenClaw, and LM Studio
Drone compute	Raspberry Pi 5 with WaveShare SX1262-class LoRa top-hat
Drone autopilot	3DR N1 connected to Raspberry Pi 5 by MAVLink
Desktop radio	Heltec WiFi LoRa 32 V3 through stable USB serial path
Field protocol	RNS/Reticulum and MeshChatX over raw LoRa unless a true LoRaWAN deployment is selected
Mapping	WebODM and supporting services under Podman
Mapping storage	/media/scottw/500GBPHOTOGRAM/
Vehicle simulation	ROS 2 Lyrical, Gazebo Sim 10.5.0, ArduPilot SITL, MAVLink, QGroundControl
Fabrication simulation	ROS 2 Lyrical, Gazebo Sim 10.5.0, robot cells, additive manufacturing, storage, kitchen, and logistics models
Ledger	Corda core behind a dedicated ledger-ingestion gateway
Archive	Local source data, signed manifests, encrypted pCloud replication, private or encrypted IPFS workflow
Monitoring	Prometheus-compatible metrics, alerts, health checks, and protected administration access
Backup	PostgreSQL/Corda-aware backup, restic or equivalent encrypted backup, and scheduled restore testing

2.2 Current Host Facts

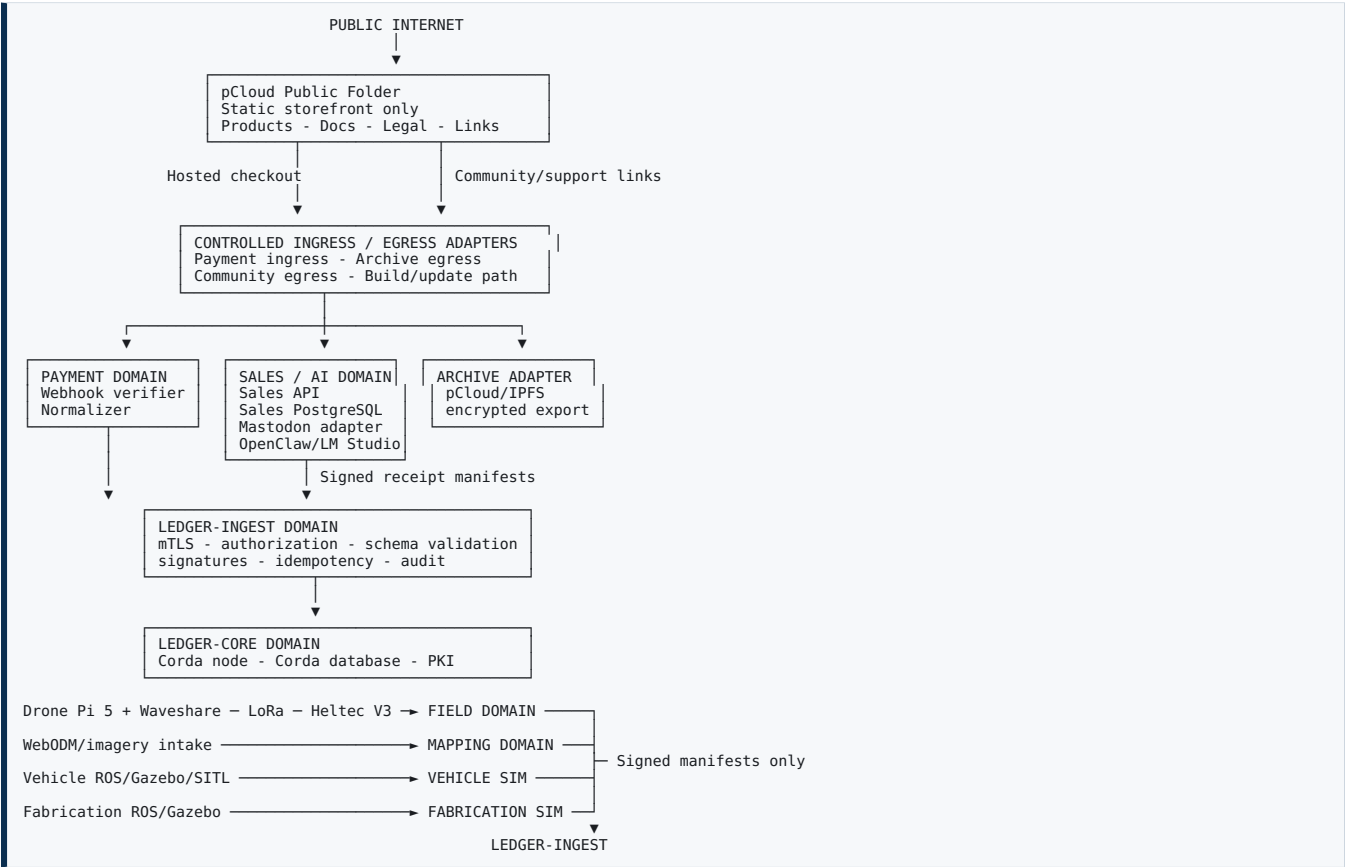
Area	Verified current value
Kernel	7.0.0-34-generic
Podman	5.7.0
Podman networks	Ten <code>ao-*</code> networks present; internal workload-domain isolation verified
GPU	EVGA NVIDIA GTX 1080
NVIDIA driver	580.178.04
NVIDIA integration	CDI devices registered, including <code>nvidia.com/gpu=0</code>
Simulation stack	ROS 2 Lyrical at <code>/opt/ros/lyrical</code> ; Gazebo Sim 10.5.0
Host PostgreSQL	PostgreSQL 18.6, loopback-only
Host Redis	Redis 8.0.5, loopback-only
Mapping drive	ext4 <code>/dev/sdb1</code> ; UUID verified; approximately 433.9 GB free of 457 GB
Mapping mount	/media/scottw/500GBPHOTOGRAM/
Current runtime model	Mixed rootless and system/rootful Podman evidence; see approved deviation section
Desktop OS	Ubuntu 26.04.1 LTS (<code>resolute</code>) userland with the Kubuntu desktop
Reticulum executable	Standalone RNS 1.4.2 available at <code>/home/scottw/.local/bin/rnsd</code> ; no standalone <code>rnsd</code> process was running during the 2026-09-24 review
Active Reticulum runtime	Embedded in the MeshChatX native backend and initialized from <code>/home/scottw/.reticulum/config</code>
MeshChatX deployment	Native headless backend at <code>/home/scottw/Applications/meshchatx-native/ReticulumMeshChatX</code>

Area	Verified current value
MeshChatX version evidence	Desktop metadata declares 4.9.1; the running backend version was not independently established during the review
MeshChatX executable verification	SHA-256 4f403e52b0a5722a49d433f23660b14b43a779fb3cc9a5a90b8f150d77f18890 matches backend-manifest.json
MeshChatX repository artifact	reticulum_meshchatx-4.8.4-py3-none-any.whl; stored in the MeshChatX repository-server identity and not the verified running artifact
MeshChatX Reticulum config	/home/scottw/.reticulum/
MeshChatX state and logs	/home/scottw/.reticulum-meshchatx/
MeshChatX local UI	127.0.0.1:18000
Reticulum public gateway listener	0.0.0.0:4242; binding verified, while firewall policy and packet reachability remain unverified

3. High-Level Architecture

3.1 Isolation Detail View

This is a **detail view of the ingress/egress portion of the master topology in ES.2** and must not contradict it. For the whole project, read ES.2.



The public storefront has no direct route to the Kubuntu host's field, mapping, simulation, database, AI, Podman, or Corda-core services.

3.2 Canonical Implementation Status

Domain or component	Architecture requirement	Current implementation state	Status	Blocking condition or next action
Host platform	Kubuntu, Podman, Quadlet, protected administration	Host inventory and base platform verified	Implemented	Maintain version matrix
Domain isolation	Separate workload networks with explicit approved paths	Ten internal workload networks and isolation test verified	Implemented	Add narrow adapters only as required
Mapping	Dedicated mapping domain and photogrammetry drive	GPU-enabled WebODM smoke test completed; orthophoto produced	Implemented with deviation	Formalize steady-state rootless/system model
Field, Reticulum, and LoRa	Raspberry Pi 5, Waveshare LoRa, Heltec V3 gateway, RNS/Reticulum, and MeshChatX	Both Heltec LoRa 32 V3/SX1262 RNodes are functional and initialized by MeshChatX; PEOPLE-RADIO uses 915 MHz/125 kHz and DRONE-RADIO uses 917 MHz/250 kHz; 32 interfaces are configured and none are explicitly disabled; RF feedback is observable on both radio bands and requires characterization; end-to-end telemetry, link-quality, and resilience acceptance tests remain pending	In progress	Measure and classify feedback on each band; record RSSI/SNR, noise floor, packet loss, airtime, retries, and cross-band isolation; complete RF telemetry and fail-safe validation

Domain or component	Architecture requirement	Current implementation state	Status	Blocking condition or next action
Vehicle simulation	Isolated ROS/Gazebo/ArduPilot SITL domain	Headless Gazebo and ROS-Gazebo bridge smoke test passed	Implemented	Add scenario and QGroundControl validation as needed
Fabrication simulation	Isolated ROS/Gazebo facility domain	Headless simulation smoke test passed	Implemented	Expand facility models and safety scenarios
Ledger	Corda core behind mTLS ingestion gateway	Corda 5.2.2 scaffolded	Blocked	Complete operator key and certificate ceremony
Sales and payment	Hosted payment flow and verified events	Sales DB deployed; payment provider and API pending	Planned	Select provider and implement verifier/API
Mastodon and OpenClaw	Restricted local support/community workflow	Local stack in progress; OAuth/client issues recorded	In progress	Complete OpenClaw and local LLM validation (Tokodon removed per ES.1)
Archive	Encrypted off-host replication and controlled IPFS workflow	Local restic backup and restore validation complete	Partially implemented	Provision pCloud/archive credentials and test replication
Backup and restore	Encrypted backup plus recurring restore testing	Encrypted restic snapshot and isolated restore test complete; recurring schedule automated 2026-08-31 (nightly restic 03:30, nightly DB dumps 03:00, weekly verify)	Implemented	Schedule recurring restore tests

3.3 DATABASES AND DATA STORES

PostgreSQL 18 is the system-wide relational database platform. The target is one host-managed PostgreSQL installation with separate logical databases and separate application roles. During the current migration, some applications still run container-scoped PostgreSQL instances; those are current-state implementations, not the permanent architecture. The database name and role boundary remain unchanged.

Target logical databases:

salesdb	# Authoritative sales/payment records
mastodon	# Mastodon application state
webodm	# WebODM/PostGIS mapping data
grafana	# Grafana application metadata, users, dashboards, datasources
metabase	# Metabase application metadata, users, questions, dashboards
cordadb	# Dedicated Corda database - NOT host PostgreSQL 18 (ES.1)
postgres	# Administrative/maintenance database

Grafana and Metabase each use PostgreSQL for their own application state. They are not treated as disposable dashboards. Their application databases require backups, restore testing, migrations, and health checks.

Grafana uses Prometheus as its operational metrics datasource and may use approved PostgreSQL datasources for business or database reporting. Metabase uses PostgreSQL source databases through dedicated read-only roles and approved views or projections.

Redis is a low-latency speed and coordination layer, not the authoritative system of record. It is used for caching, queues, locks, task brokering, and transient operational coordination. Important business, payment, user, dashboard, and application metadata remain in PostgreSQL.

Prometheus TSDB remains separate as the specialized time-series store for metrics. It is not replaced by PostgreSQL, Grafana, or Metabase.

SQLite and H2 are not approved application databases for Grafana or Metabase. They may exist only in unrelated desktop/browser applications where embedded storage is required, or in retained migration backups.

Database/ledger authority and status legend

- **AUTHORITATIVE RELATIONAL:** PostgreSQL owns detailed operational data, searchable business records, and reporting projections.
- **AUTHORITATIVE LEDGER:** Corda owns final sale/contract, receipt association, entitlement, and approved ledger state transitions.
- **ACTIVE:** Currently running and verified.
- **CURRENT MIGRATION STATE:** Container-scoped PostgreSQL may still exist for an application while its data is being consolidated into the host PostgreSQL platform.

- **TARGET:** Documented end-state architecture, not yet fully deployed.
- **BLOCKED:** Requires an operator key/certificate, credential, service, or acceptance step.

Do not treat a **TARGET**, **CURRENT MIGRATION STATE**, or **BLOCKED** component as active production state.

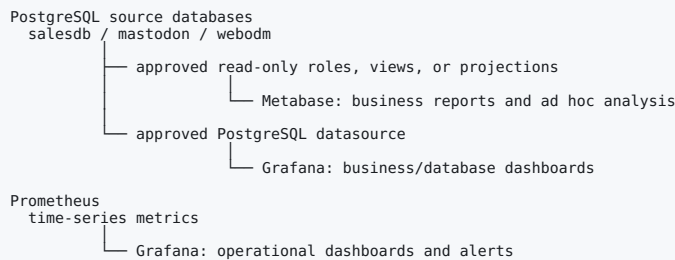
3.3.1 Program-to-Database Map

The following table groups the currently identified programs by the database software they use. It is the starting point for discussing reporting and data integration; it is not a list of every installed package or desktop settings module.

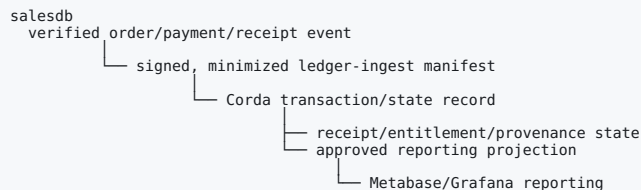
Database software	Software/program	Database name or store	Current role and reporting value
PostgreSQL 18	Host PostgreSQL service	Host cluster; <code>grafana</code> ; <code>metabase</code> ; <code>postgres</code>	Shared relational platform and administrative/maintenance cluster
PostgreSQL 18	Grafana	<code>grafana</code>	Grafana users, dashboards, folders, datasource definitions, preferences, and alert state
PostgreSQL 18	Metabase	<code>metabase</code>	Metabase users, collections, questions, dashboards, database connections, and settings
PostgreSQL 18	Mastodon web/Sidekiq	<code>mastodon</code> in <code>mastodon-db</code>	Accounts, posts, media metadata, federation state, and background-job application data
PostgreSQL/PostGIS	WebODM web/worker	<code>webodm</code> or <code>webodm_dev</code> in <code>db</code>	Mapping projects, processing state, users, and geospatial data
PostgreSQL/PostGIS	NodeODM	WebODM PostgreSQL plus filesystem processing data	Processing-node state and coordination; large image/output artifacts remain filesystem data
PostgreSQL 18	Sales database service	<code>salesdb</code> in <code>sales-db</code>	Target source for customers, orders, products, payments, receipts, entitlements, and audit history. The repository schema is defined; the live <code>salesdb</code> application schema still requires explicit initialization.
PostgreSQL 18	Corda node, when activated	<code>cordadb</code> (separate logical database in the host PostgreSQL 18 cluster, per ES.1)	Correct Corda persistence per ES.1. The node was scaffolded on H2 by mistake and must be migrated onto <code>cordadb</code> ; it is not yet active and still requires the operator key/certificate ceremony (section 18.2)
Redis 8	Host Redis service	Host Redis database 0	General low-latency cache/coordination layer; no current application data confirmed
Redis 8	Mastodon cache/queue service	<code>mastodon-redis</code> database 0	Cache, queues, and background-job coordination; not authoritative business data
Redis 8	WebODM broker	<code>broker</code> database 0	Celery/task broker and worker coordination; not authoritative mapping data
Prometheus TSDB	Prometheus	<code>/prometheus</code> persistent volume	Time-series metrics, service health, resource usage, and operational monitoring
Prometheus TSDB	Grafana metrics datasource	Prometheus at <code>ao-prometheus:9090</code>	Grafana visualizes metrics; Prometheus remains the separate metrics store
SQLite	Akonadi/KDE PIM applications	Akonadi SQLite data	Contacts, calendars, mail indexes, and local personal-information data
SQLite	Firefox, Brave, Chrome, and Edge	Browser profile SQLite stores	Browser history, site storage, caches, certificates, and profile data
SQLite	Podman	Rootless container metadata store	Container, image, network, and volume metadata; not application data
SQLite	Selected ROS/local tools	Application-specific local SQLite files	Local tool state where enabled; not a shared reporting source
Filesystem/local metadata	LM Studio, OpenClaw, MeshChatX, QGroundControl, ArduPilot, Gazebo, and simulation tools	Application files, logs, project files, and local state	Operational or engineering data that is not automatically part of SQL reporting
H2	Grafana/Metabase legacy migration data	Retained H2 backup files only	No longer active; retained temporarily for rollback and migration evidence

3.3.2 Data Flow into Reporting and Ledger Records

The intended reporting flow is:



Corda is not a replacement for the source PostgreSQL databases. The intended flow is:



Use stable correlation fields in the source-to-ledger integration, including a transaction/order reference, serial number where applicable, UTC timestamp, event type, status, and content hash. Do not copy payment credentials, private keys, or unrestricted customer datasets into Corda. Corda remains authoritative for approved ledger/provenance state, while PostgreSQL remains authoritative for domain-operational source data.

Before building sales reporting, initialize and verify the `salesdb` schema and define read-only reporting views. Before enabling blockchain-related flows, complete the Corda key/certificate ceremony and implement the ledger-ingest manifest, correlation-ID, signature, idempotency, and audit requirements.

4. Security, Isolation, and Data Policy

4.1 Non-Negotiable Rules

1. Inspect before changing.
2. Preserve existing data.
3. Never format, repartition, delete, prune, or overwrite without explicit operator approval.
4. Never install Docker daemon, Docker Compose, or Watchtower.
5. Use Podman and Quadlet only.
6. Never expose a public port without explicit operator approval.
7. Never place secrets in scripts, logs, HTML, Git, pCloud Public Folder, IPFS, Corda payloads, shell history, or documentation examples.
8. Never use `--privileged` as a default.
9. Use pinned image digests for operational services.
10. Verify the photogrammetry drive before deploying or operating WebODM.
11. Record commands, versions, significant output, and failures in the installation or operational journal.
12. Stop and report conflicts involving services, packages, networks, mounts, ports, serial devices, firewall policy, or existing data.
13. Do not broaden network access, database privileges, filesystem access, container privileges, or secret access merely to bypass an error. A documented local integration path with least-privilege credentials is permitted when it is required for PostgreSQL reporting, backup, health checking, or application migration.
14. Require explicit human approval before publishing external communications, initiating payments, changing production credentials, deleting data, or modifying external records.

4.2 Data Classification

Classification	Examples	Handling requirement
Public	Storefront HTML, intentionally published documentation, approved product data	May be placed in pCloud Public Folder
Internal operational	Non-sensitive configuration, health data, non-sensitive manifests, unit status	Restricted local access; do not publish by default
Sensitive	Customer contact data, payment references, precise telemetry, sensitive imagery, proprietary technical designs	Domain-restricted storage; encrypted backup; no public IPFS
Secret	Passwords, tokens, API keys, private keys, Corda keystores, archive credentials, radio keys	Podman secrets, systemd credentials, or approved secret files only

4.3 Prohibited Paths

Sales/AI → MAVLink, ArduPilot, ROS, Gazebo, LoRa, RNS, MeshChatX
Sales/AI → WebODM workers, raw imagery, Corda core
Payment → OpenClaw, LM Studio, Mastodon, field, mapping, simulation
Field → payment provider, Mastodon, OpenClaw, LM Studio, Corda core
Mapping → flight control, LoRa/RNS, payment provider, Mastodon, Corda core
Vehicle simulation → live drones, live radios, sales, payments, Corda core
Fabrication simulation → live machinery during phase one, sales, payments, Corda core
Public internet → PostgreSQL, Redis, WebODM workers, LM Studio, Corda, ROS, MAVLink, Gazebo, QGroundControl, RNS, MeshChatX

4.4 Approved Internal Paths

Sales receipt manifest	→ Ledger-ingestion gateway
Verified payment event	→ Sales API and/or ledger-ingestion gateway
Field telemetry manifest	→ Ledger-ingestion gateway
Mapping deliverable manifest	→ Ledger-ingestion gateway
Vehicle simulation manifest	→ Ledger-ingestion gateway
Fabrication simulation manifest	→ Ledger-ingestion gateway
Ledger receipt or entitlement status	→ Authorized service through narrow API
Signed mission release	→ Field mission-release service
Validated image set	→ WebODM intake service

All cross-domain requests require:

- Mutual TLS.
- A dedicated service certificate or identity.

- Signed payload where durable provenance is required.
- Schema validation.
- Timestamp and nonce or equivalent replay defense.
- Durable idempotency key handling.
- Audit record.
- Explicit authorization policy.

Mutual TLS authenticates transport peers. Detached manifest signatures permit independent verification after storage, export, or audit. These are separate controls and should be used together for provenance-bearing artifacts.

5. Network Domains and Controlled External Access

5.1 Workload Domains

Podman network	Purpose	Public exposure	Permitted output
ao-sales	Sales API, sales PostgreSQL, Mastodon adapter, OpenClaw, LM Studio	No direct public exposure	Signed order, receipt, and entitlement manifests
ao-payment	Provider webhook verifier and payment adapter	No direct public exposure	Verified normalized payment state
ao-field	Heltec gateway, RNS/MeshChatX, telemetry spool, mission-release service	No direct public exposure	Signed telemetry and mission manifests
ao-mapping	WebODM, NodeODM, Redis, mapping DB, imagery intake/exporter	Operator/VPN access only when approved	Signed mapping deliverable manifests
ao-sim-vehicle	ROS 2, Gazebo, ArduPilot SITL, MAVLink, QGroundControl simulation	No direct public exposure	Signed vehicle-simulation manifests
ao-sim-fabrication	ROS 2, Gazebo, robot cells, additive manufacturing, facility model	No direct public exposure	Signed fabrication-simulation manifests
ao-ledger-ingest	mTLS validation gateway, authorization, audit, idempotency	No direct public exposure	Corda receipt IDs and status
ao-ledger-core	Corda node, Corda database, certificate/keystore material	No direct public exposure	No direct output
ao-data	Narrow controlled data plumbing where unavoidable	No direct public exposure	Controlled references only
ao-admin	Monitoring, backup, restore validation, administration	VPN or explicitly allowlisted administration only	Operational reports

All workload-domain networks are `Internal=true`. CIDRs are recorded in:

```
/ALWAYSON/config/platform/network-cidrs.yaml
```

No workload service may receive unrestricted Internet access simply by joining its application-domain network.

5.2 Controlled Ingress and Egress Adapters

External connectivity is allowed only through narrowly scoped, independently reviewed adapters. These adapters are architecture-controlled exceptions, not general-purpose Internet access.

Adapter/network	Purpose	Direction	Mandatory controls
ao-ingress-payment	Payment-provider webhook receiver or approved relay	Inbound	Minimal listener, provider-signature verification, rate limits, audit log, normalized event output
ao-egress-archive	Encrypted pCloud replication and approved IPFS operations	Outbound	Destination allowlist, TLS validation, encrypted payloads, separate credentials, transfer audit
ao-egress-community	Approved Mastodon/community activity when remote connectivity is explicitly enabled	Outbound	Approved host allowlist, minimum OAuth scope, rate limits, publication approval log
ao-build-update	Image and package acquisition before controlled promotion	Outbound	Verified source, digest capture, update audit, no direct workload attachment

No sales, mapping, field, simulation, database, AI, or ledger-core container may attach directly to an Internet-capable network. An external adapter must use separate credentials, destination allowlists, validated DNS/TLS, firewall policy, minimal permissions, and connection logging.

5.3 Approved Local Data Paths

The following local paths are normal integration paths and do not require a new architecture decision:

- Application containers to their approved PostgreSQL database endpoint.
- Metabase and Grafana to PostgreSQL through dedicated roles, views, or approved reporting interfaces.
- Grafana to Prometheus for operational metrics.

- Host administration and backup jobs to PostgreSQL through loopback or an explicitly documented local bridge.
- Application workers to their required Redis queue or broker.

These paths must not become public listeners, must use separate credentials, and must not grant unrelated applications access to each other's owner, migration, backup, payment, or ledger credentials. Network isolation remains a defense-in-depth control; PostgreSQL roles and grants are the primary authorization boundary for database access.

6. Component Boundaries, GUI Reporting Tools, and Operator Access

6.1 Component Boundary Matrix

Component	Owning domain	Inputs accepted	Outputs allowed	Persistent data	External connectivity
Sales API	ao-sales	Verified payment state and approved support requests	Signed receipt/entitlement manifests	Sales PostgreSQL	None directly
Payment verifier	ao-payment	Provider webhook or approved relay event	Verified normalized payment event	Minimal event and audit record	Through ao-ingress-payment only
Mapping intake	ao-mapping	Authenticated imagery upload	Validated image-set reference	Intake, validation, quarantine record	None directly
WebODM/NodeODM	ao-mapping	Validated mapping task input	Processing output to mapping exporter	Dedicated photogrammetry volume	None directly
Field gateway	ao-field	USB serial LoRa frames	Normalized telemetry manifest	Raw packet store and telemetry spool	USB serial and radio only
Vehicle simulator	ao-sim-vehicle	Approved scenario/model artifact	Signed simulation manifest	Vehicle simulation data path	None directly
Fabrication simulator	ao-sim-fabrication	Approved facility/task model	Signed simulation manifest	Fabrication simulation data path	None directly
Ledger ingestion	ao-ledger-ingest	Signed mTLS manifests	Receipt/status response	Audit and idempotency state	Only to ledger core
Ledger core	ao-ledger-core	Ledger-ingestion gateway requests only	No direct public output	Corda state and PKI	None directly
Archive adapter	ao-egress-archive	Approved encrypted archive bundle	Replication result/status	Staging and transfer log	Outbound only
Community adapter	ao-egress-community	Approved publication or support request	Remote delivery/status response	Publication audit log	Outbound only

6.A GUI Reporting Tools and Podman Network Mapping

This subsection is an **architecture requirement**. It defines the required relationship between operator GUIs, reporting tools, dashboards, desktop clients, external provider dashboards, workload domains, and Podman networks.

Section 19 records what remains to be implemented, documents, and validated for this requirement. It does not redefine, weaken, or replace it.

An **associated domain** identifies the operator workflow a tool serves. It does not grant broad Podman-network membership, database access, host access, shared storage, shared credentials, or cross-domain control. A host desktop application, host browser, or external provider dashboard has no Podman network attachment unless it is itself implemented as a container attached to that network.

`ao-admin` is the protected administration, monitoring, and reporting plane. It may host Grafana for operational dashboards and alerts, Metabase for FOSS accounting/database-heavy reporting, and narrowly authorized administration tools. It must not become a shared universal network. `ao-data` remains narrow controlled data plumbing, not a default GUI, shared-database, or reporting network.

6.A.1 GUI ↔ Podman Network Mapping

Attachment rule (applies to every row below). Each GUI or workflow is attached to exactly **one** owning `ao-*` network and is denied attachment to all the others. The denied set is always the complement of the owning network, so it is not repeated per row. Read-only access to a second network is permitted **only** where the Approved access path column says so explicitly, and never as a broad membership.

#	GUI / workflow	Owning network	Approved access path	Status
1	Mastodon web / Konqueror client	ao-sales	Approved localhost Mastodon web/streaming origin; loopback-only publication when enabled.	In progress; local service model exists. OpenClaw/OAuth validation remains open (section 19.3).
2	WebODM browser UI	ao-mapping	Approved loopback WebODM listener; VPN/authenticated access only if separately approved.	Implemented with deviation; smoke test and UI path verified. Final rootless/system/mixed designation remains required.
3	QGroundControl simulation client	ao-sim-vehicle	Approved local SITL/MAVLink-router endpoint; ROS_DOMAIN_ID=21; GZ_PARTITION=alwayson_vehicle_sim.	Planned GUI workflow; headless vehicle simulation and ROS-Gazebo bridge verified.
4	Gazebo visualization — vehicle	ao-sim-vehicle	Approved vehicle ROS/Gazebo visualization path; separate DDS/interface policy remains required.	Planned GUI; headless runtime verified.
5	Gazebo visualization — fabrication	ao-sim-fabrication	Approved fabrication ROS/Gazebo visualization path; ROS_DOMAIN_ID=22; GZ_PARTITION=alwayson_fabrication_sim.	Planned GUI; headless runtime verified.
6	LM Studio / OpenClaw support chat	ao-sales (host-local LM Studio) / ao-sales (containerized OpenClaw)	Host desktop use; approved loopback inference endpoint or narrow authenticated bridge only.	In progress; endpoint and container/host boundary require formalization (section 19.3).
7	Grafana operational monitoring dashboard	ao-admin	VPN or authenticated, allowlisted administration access only.	Planned; implements Section 17.2 operational monitoring and alerting requirements.
8	Metabase accounting, sales, and database-heavy reporting GUI	ao-admin	VPN or authenticated, allowlisted administration access. Dedicated least-privilege reporting identities use approved loopback/tunnel/bridge paths.	Planned. Implement after reporting views, reporting identities, and sales/payment workflow are approved.
9	Sales, receipt, fulfillment, entitlement, return, and approved support reporting	ao-admin	Metabase uses approved read-only reporting views and identities. DBeaver uses an explicit purpose-limited loopback or approved tunneled connection.	Planned after sales API, payment verifier, reporting schema/views, and payment-provider workflow are implemented.
10	Ledger provenance, receipt, entitlement, approval, release, and ingestion reporting	ao-admin	VPN or authenticated administration access. Metabase reads approved reporting views/projections through a dedicated reporting identity; Grafana receives supported metrics/status only.	Planned/blocked pending key/certificate ceremony, Corda status/metrics configuration, and approved reporting projection.
11	Backup/restore status display	ao-admin	Same protected administration boundary as Grafana and Metabase.	Planned; backup and isolated restore evidence already exist.
12	Field gateway / link-quality display	ao-field	Approved USB serial/local diagnostic display or protected Grafana dashboard.	In progress; Heltec V3 connection and stable serial path verified 2026-08-31; gateway service deployment pending (section 19.4).
13	Ledger/Corda console and maintenance	ao-ledger-ingest	Narrow approved operator-management path after key/certificate ceremony; no public access.	Blocked pending Section 18.3 ceremony and current ledger backend diagnosis.
14	PostgreSQL reporting, schema inspection, and controlled administration	host loopback	Explicit loopback or approved narrow tunnel/bridge using a dedicated least-privilege database identity.	Planned. PostgreSQL is loopback-only; formal reporting/maintenance roles and views are required.

#	GUI / workflow	Owning network	Approved access path	Status
15	Redis diagnostic client	ao-data (optional)	Explicit loopback or approved narrow diagnostic path using a scoped Redis ACL identity.	Optional/planned only if diagnostic value justifies deployment.
16	Payment-provider dashboard	provider-hosted	Provider-authenticated browser workflow.	Blocked/open pending payment-provider selection.
17	No GUI — controlled data services	ao-data (narrow only)	Host services remain loopback-only; administration/reporting uses dedicated host or ao-admin identities and paths.	Implemented as intentional GUI-less controlled plumbing.
18	No GUI — payment verifier	ao-payment	Provider-hosted checkout and provider dashboard; local verifier has no GUI.	Blocked/open pending provider decision under Section 18.4.

6.A.2 Reporting Tool Roles

Tool	Primary purpose	Mandatory boundary
Metabase	FOSS relational reporting: sales, orders, receipts, fulfillment, entitlements, returns, approved support summaries, ledger/provenance projections, saved questions, dashboards, filters, and exports	Runs in ao-admin; uses PostgreSQL for its own application state and connects to approved source databases through dedicated read-only roles, views, or projections. It never receives superuser, database-owner, migration, backup, payment-provider, or Corda-key credentials.
Grafana	Operational monitoring and visualization: metrics, service health, alerts, queue depth, latency, resource use, storage, GPU state, backup age, restore-test status, certificate expiry, ingest failures, and approved PostgreSQL business/database metrics	Runs in ao-admin; uses PostgreSQL for its own application state, Prometheus for metrics, and approved PostgreSQL datasources for business/database reporting. It never becomes a shell, container-management, or control path.
Corda management/ API/CLI	Corda lifecycle, configuration, certificate-aware administration, and controlled maintenance	Uses a documented narrow management path after the required ceremony; it is not replaced by Metabase or Grafana.
DBeaver / optional pgAdmin	Exceptional SQL analysis, schema inspection, backup/restore validation, and controlled database maintenance	Uses an explicit least-privilege identity and loopback or approved narrow tunnel/bridge; it is not the routine accounting/reporting surface.
Payment-provider dashboard	Provider-authoritative charges, refunds, disputes, payouts, exports, and reconciliation	External provider service; no Podman network attachment and no replacement of local verified-event controls.

Metabase may report on approved Corda-derived business and provenance data only through a deliberate read-only reporting projection, approved views, supported status interface, or ledger-ingestion audit/status records. Metabase must not become the primary interface to Corda internal persistence tables, administer Corda, receive Corda private keys/keystores, or create a broad route into ao-ledger-core.

6.A.3 Conformance Requirements

All current and future GUI, dashboard, reporting, database-administration, and operator-access implementations must comply with this subsection and Sections 4, 5, 14, and 17.

- Every tool must have a named operator purpose, actual runtime placement, approved data/status source, documented access path, and explicit implementation status.
- Every containerized GUI must have documented Podman-network membership, listener policy, service owner, image digest, authentication method, and least-privilege identity.
- Every host desktop GUI and provider dashboard must be recorded as having no Podman network attachment unless it is actually containerized.
- Reporting identities must enforce read-only access to source databases or services. This does not make Grafana or Metabase read-only applications: each owns a separate PostgreSQL application database and role.
- ao-admin receives approved PostgreSQL reporting, exporter, status, projection, API, relay, tunnel, or push paths. It must not join every workload network.
- ao-data is not a shared unrestricted database, general-purpose shell, or authorization bypass. It may carry narrowly approved local data paths.
- No GUI may add a public listener, broad host networking, unrestricted Podman socket access, --privileged, shared writable storage, or unrelated-domain secret merely to simplify deployment or troubleshooting.
- Any material deviation requires an approved deviation record under Section 18 before production declaration.

The machine-readable implementation inventory for this requirement is:

```
/ALWAYSON/config/platform/gui-boundary-matrix.yaml
```

7. Public Storefront and Payment Policy

7.1 Storefront Boundary

The public storefront is static HTML hosted in the pCloud Public Folder. HTML project github: <https://github.com/300x3/HTML-300X3> Static HTML output folder for pCloud: `public/html/` (generated by `scripts/build-html.mjs`; dependency-free pages: index, buildings, vehicles, equipment, maps, discussion, documentation, donate).

7.1.1 Frontend Website Details

Source: HTML-300X3 repo (React + TanStack Router; static export in `public/html/`). Navigation is defined in `src/lib/nav.ts` (NAV sections with modal previews, pCloud folder links, and SketchUp/Trimble model links). The pCloud Public Folder mirrors the static export layout.

Top-level sections and catalog items (modals):

Equipment (`/equipment`)

- Adapter (soda threads to 0.5" NPT — "TUBER").
- Boiler (water boiler, power production, chemistry set).
- Pneumatic Speargun Ulu (Damascus ulu + forearm pneumatic speargun).
- Structural Battery ("power sandwich" gas/liquid tank + battery case).
- Appliances (12oz micro-appliances — "app cans").
- Computer (12oz-can Raspberry Pi case + wireless HDMI/video-glasses kit).
- Camping (shopping-list discussion, pricing, where-to-buy).

Buildings (`/buildings`)

- Furniture.
- ADU (80sf and up).
- Mall.
- Tower.
- Concrete Island.

Vehicles (`/vehicles`)

- Drone (air/land/sea).
- Boat (micro modular aircraft carrier).
- Personal Vehicle.
- Electric Car Wheel.
- Balloon.

Supporting sections: Digital (images, topography/3D points, "Where's My ___?", route-around-your-county), Discussion (Mastodon forum + 300X3@POSTEO.NET), Documentation (intro video, working project-plan PDF, server coding, 3D models, HUD app, simulations, AI/hardware/software/fabrication/raw-material links), Donate (PayPal hosted button + Zelle + card/other).

Sales-link integration plan

Each catalog modal under Equipment / Buildings / Vehicles gets a sales action that stays inside the static-site boundary (no secrets, no local ports, no internal hosts — see Section 7.1 prohibitions):

1. Modal shows product images, parts list / detailed drawings link, pCloud folder link, and IPFS digital-asset mark where applicable.
2. A purchase button routes to provider-hosted checkout (PayPal hosted button today; Zelle instructions and Coinbase/USDC flow per Section 18.4 as implemented) or to a `mailto:300X3@POSTEO.NET` order-request template carrying product name, options, and quantity.
3. Checkout completion returns a provider-signed event (or manual reconciliation record for Zelle/wire) into the Section 7.3 sales and receipt sequence; Corda records the receipt/entitlement state per Section 11.
4. No payment-card data, webhook secrets, OAuth tokens, or ledger keys ever appear in the static HTML, pCloud folder, or Git history.

```

pCloud Public Folder
├── index.html
├── products/
├── catalog/
├── support/
├── community/
├── legal/
│   ├── privacy.html
│   ├── terms.html
│   ├── returns.html
│   └── shipping.html
├── assets/
│   ├── css/
│   ├── js/
│   ├── images/
│   └── downloads/

```

The public site may include:

- Product catalog and documentation.
- Hosted payment checkout links.
- Provider-controlled payment buttons.
- Order follow-up and support links.
- Mastodon/community links.
- AI-assisted support entry points that do not expose private infrastructure.
- Shipping, return, warranty, privacy, and legal content.

The public site must never include:

- Payment-provider secret keys.
- Corda keys, RPC credentials, or node addresses.
- Mastodon OAuth tokens.
- pCloud archive credentials.
- IPFS private keys or swarm keys.
- Local hostnames, LAN addresses, Podman ports, or private API routes.
- Database connection strings.
- Drone radio configuration, control endpoints, or flight-control access.
- Internal service certificates, identifiers, or diagnostic output.

7.2 Payment and Settlement Policy

ALWAYS ON does not process, transmit, or store payment-card numbers, CVV values, or payment-provider secret material in the storefront, sales database, Corda, Git repository, logs, pCloud Public Folder, or IPFS.

The default payment model is provider-hosted checkout. The selected provider is responsible for payment-card capture and authorization. The local payment-verifier service accepts only provider-signed webhook events and stores normalized business state.

Payment method	Intended use	Required control
Hosted card checkout	Standard online transactions	Provider-hosted checkout, signature-verified webhook, no local card handling
Hosted PayPal checkout	Optional provider-supported checkout	Provider-controlled flow and verified event
Wire transfer	Approved high-value transactions	Manual reconciliation, operator approval, auditable reference record
Other payment methods	ARE TO BE PROCESSED BY CORDA - SPECIFICALLY STABLECOIN IN RELATION TO COINBASE OR SIMILAR	Requires documented provider terms, accounting treatment, refund process, and explicit operator approval

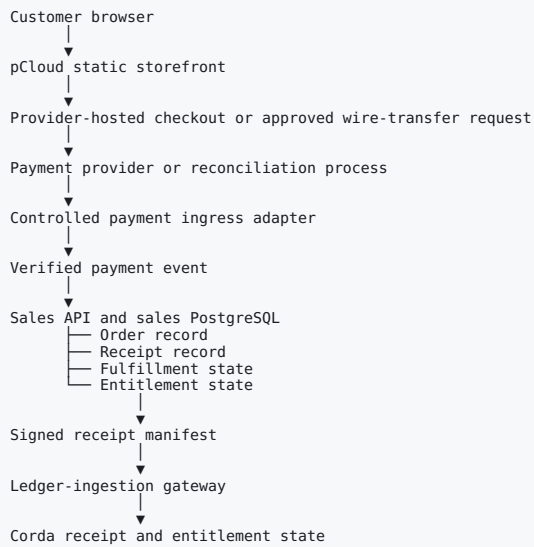
Corda is the central source of truth for financial ledger information, correlated to the PostgreSQL operational database per transaction and per serial number. Corda does not accept payment cards and does not replace the payment provider, banking, tax, consumer-protection, accounting, or refund processing.

Corda does record approved receipt, fulfillment, entitlement, or provenance state after a payment event has been verified or manually reconciled. It must include the related transaction data typical of the financial and payments industry, including the ledger details, and must be queryable by the authorized reporting service (Metabase; Grafana where enabled).

PayPal, Zelle, and stablecoin (USDC or similar) are the standard payment methods for the project. Corda must verify payments made with them while documenting those transactions within a secure blockchain and ledger.

Approved 2026-08-28 (section 18.4): Zelle processing is the same as PayPal and Coinbase/stablecoin processing. The existing authentication method for each payment service is to be verified by Corda in some manner.

7.3 Sales and Receipt Sequence



8. Mapping and Photogrammetry

8.1 Dedicated Storage

The dedicated local workspace for WebODM and photogrammetry is:

```
/media/scottw/500GBPHOTOGRAM/
```

This drive is authoritative for:

- Incoming drone imagery.
- Validated imagery sets.
- Rejected and quarantined uploads.
- WebODM media and project data.
- NodeODM intermediates.
- Mapping deliverables.
- Mapping processing and provenance manifests.
- pCloud/IPFS archive staging.
- Mapping-database backup exports.

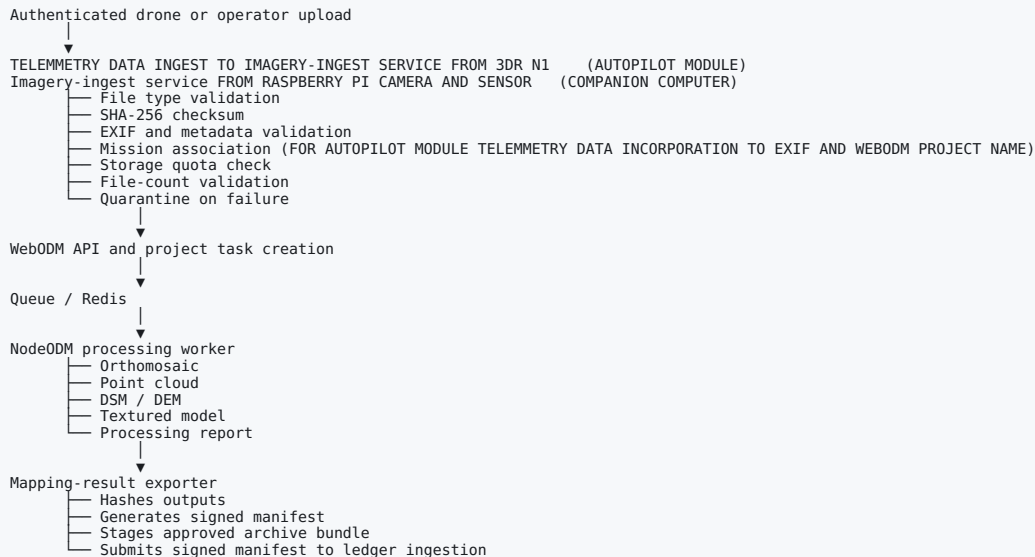
WebODM must not use the root filesystem, `$HOME`, or Podman writable container layers for high-volume processing.

8.2 Required Directory Tree

```
/media/scottw/500GBPHOTOGRAM/
├── README.md
├── .mounted-ok
├── incoming/
│   ├── drone/
│   ├── operator/
│   └── quarantine/
├── validated/
│   └── <mission-id>/
├── rejected/
│   └── <mission-id-or-date>/
├── webodm/
│   ├── media/
│   ├── projects/
│   ├── nodeodm/
│   ├── temp/
│   └── logs/
├── deliverables/
│   └── <mission-id>/
│       ├── orthophoto/
│       ├── point-cloud/
│       ├── dem-dsm/
│       ├── textured-model/
│       ├── reports/
│       └── manifest/
├── manifests/
│   ├── intake/
│   ├── processing/
│   └── ledger-submissions/
├── exports/
│   ├── pcloud-staging/
│   └── ipfs-staging/
├── backups/
│   └── mapping-db/
├── retention/
│   ├── pending-review/
│   └── eligible-for-archive/
├── tmp/
└── processing/
```

No directory in this tree may be world-writable. Use dedicated mapping ownership, explicit groups, and ACLs only when necessary.

8.3 Mapping Processing Flow



8.4 Persistent Locations

Data	Location
Raw images	/media/scottw/500GBPHOTOGRAM/incoming/
Validated images	/media/scottw/500GBPHOTOGRAM/validated/
WebODM media/projects	/media/scottw/500GBPHOTOGRAM/webodm/
Intermediate work	/media/scottw/500GBPHOTOGRAM/webodm/nodeodm/ and tmp/
Deliverables	/media/scottw/500GBPHOTOGRAM/deliverables/
Mapping PostgreSQL	/ALWAYSON/data/mapping/postgres/ or approved Podman volume
Redis persistence	/ALWAYSON/data/mapping/redis/
Signed manifests	/ALWAYSON/artifacts/mapping-manifests/

8.5 Mapping Mount Validation

The drive must be identified by filesystem UUID, not by `/dev/sdX`.

WebODM must refuse to start when:

- The mount is absent.
- The mountpoint resolves to the root filesystem.
- The mounted UUID differs from the approved UUID.
- `.mounted-ok` is absent.
- Available space is below the configured minimum.
- Required directories are missing.
- Mapping service ownership or permissions are incorrect.

Required validation:

```
lsblk -f
findmnt /media/scottw/500GBPHOTOGRAM
blkid
df -hT /media/scottw/500GBPHOTOGRAM
```

Begin with CPU-only validation. Enable GTX 1080 access only after validated container GPU runtime, driver compatibility, measurable workload benefit, and a documented CPU-only recovery path.

8.6 3D Model Identity and Database Cross-Referencing

Every 3D model, model revision, component, assembly, and derived artifact must be addressable from the same database and ledger correlation system used for sales and receipts. The model file is not itself the authority; the authoritative relationship is the PostgreSQL registry entry plus the signed content manifest.

8.6.1 Identifier hierarchy

Use a stable, globally unique `model_object_id` for the logical object and a separate `model_revision_id` for each version:

```
model_object_id    # Stable identity of the logical 3D object or assembly
model_revision_id  # One specific model revision/artifact
serial_number      # Physical asset, when the model represents a sold product
correlation_id     # Business/event correlation across PostgreSQL and Corda
receipt_number     # Commercial receipt, when the model is sold
event_timestamp_utc # When the relationship/event was recorded
content_hash_sha256 # Hash of the exact model file or packaged artifact
```

`model_object_id` remains stable across revisions. A revised model must not reuse an old revision ID. `serial_number` links the model to a physical product; it is not a replacement for the model object ID.

8.6.2 Metadata carried with the 3D model

Each model package must carry a sidecar metadata document or embedded metadata block containing at least:

```
{
  "model_object_id": "OBJ-300X3-BATTERY-0001",
  "model_revision_id": "REV-2026-09-25-01",
  "object_type": "cad_assembly",
  "source_system": "cad_release",
  "serial_number": "SN-300X3-000042",
  "correlation_id": "ORDER-2026-000123-A",
  "receipt_number": "RCPT-2026-000123",
  "event_timestamp_utc": "2026-09-25T12:34:56Z",
  "schema_version": "1.0",
  "content_hash_sha256": "SHA256_DIGEST",
  "source_artifact_reference": "opaque internal reference",
  "license_reference": "approved license/terms reference",
  "is_public_proof_eligible": false
}
```

The metadata is cross-referenced, not duplicated wholesale: the model contains identity and reference fields, PostgreSQL contains the operational record, and Corda contains the signed state/reference.

8.6.3 Database registry

The model registry should be implemented in PostgreSQL with tables equivalent to:

```
model_objects
model_object_id, object_type, canonical_name, created_at_utc, created_by,
current_revision_id

model_revisions
model_revision_id, model_object_id, revision_number, content_hash_sha256,
source_artifact_reference, archive_reference, license_reference,
created_at_utc, created_by

model_object_links
model_object_id, link_type, serial_number, correlation_id, receipt_number,
event_timestamp_utc, valid_from_utc, valid_to_utc

model_ledger_references
model_revision_id, corda_event_type, corda_transaction_id, corda_state,
corda_confirmed_at_utc, manifest_reference
```

`model_object_links` is the cross-reference table. It relates a model object to a product, serial number, receipt, order, mapping project, simulation result, release, or other approved object without embedding the operational record in the 3D file.

8.6.4 Cross-reference flow

```
CAD/3D authoring tool
  |
  +-- model_object_id + model_revision_id
  |
  v
Model registry (PostgreSQL)
  |
  +-- serial_number -> product/asset record
  |
  +-- correlation_id + receipt number -> sale contract record
  |
  +-- content hash -> exact model artifact
  |
  +-- manifest reference -> signed ledger event
  |
  v
Corda state
```

A viewer, CAD tool, WebODM/NodeODM exporter, or marketing application resolves a model by reading its object/revision IDs, validating the content hash, looking up the PostgreSQL registry, following approved links to the serial/receipt record, following the Corda projection, and returning only fields permitted for that audience.

8.6.5 Integrity and relationship rules

- A model revision has exactly one `model_revision_id`.
- A model revision has one immutable `content_hash_sha256`.
- A revision ID cannot point to different content hashes.

- A model object may have many revisions, but only one current revision.
- A physical serial number may have many model revisions over its lifetime.
- A receipt may reference many model objects or serials through the link table.
- A model relationship records `event_timestamp_utc` and its source.
- Superseded revisions are preserved and marked `superseded`, never reused.
- A Corda reference is required before a model is called provenance-verified.
- A content hash proves file integrity, not authenticity or publication safety.

8.6.6 Public and private model metadata

Internal metadata may contain serial numbers, correlation IDs, and opaque references. Public marketing metadata should contain only approved fields:

```
model_object_id or public proof ID
product/SKU
approved serial or proof token
revision label
provenance status
public verification reference
content hash or public proof hash
license/terms reference
```

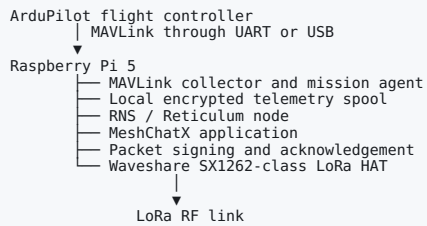
Do not publish customer identity, receipt totals, addresses, payment references, private simulation data, internal paths, or Corda transaction details unless that disclosure is explicitly approved.

8.6.7 Relationship to the sale/receipt process

For a sold product, the 3D model metadata (`model_object_id`, `model_revision_id`, `serial_number`) resolves to the PostgreSQL model registry, `sale_contract_lines`, receipt/correlation projection, and signed Corda provenance reference. The receipt and model may each show a reference to the same correlation record. Neither file is the authoritative sale ledger.

9. Field and LoRa Architecture

9.1 Drone-Side System



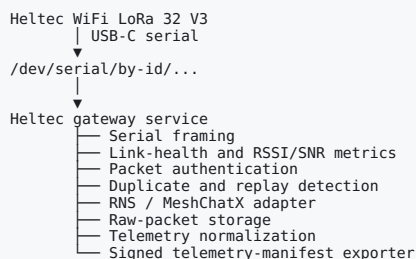
9.2 Desktop Gateway

9.2.1 MeshChatX Local Service Port

The desktop MeshChatX application uses the dedicated loopback port `https://127.0.0.1:18000` for its native backend and local web UI. This port is separate from the ALWAYS ON mapping service listener on `127.0.0.1:8000`.

Service	Domain	Listener	Exposure	Ownership
MeshChatX native backend / web UI	Field / Reticulum	<code>https://127.0.0.1:18000</code>	Loopback only	<code>scottw</code> user service
WebODM web service	Mapping / ao-mapping	<code>127.0.0.1:8000</code>	Loopback only	<code>ao-webodm-web.service</code>
Reticulum transport	Field / Reticulum	Reticulum-configured interfaces	No HTTP listener	Embedded MeshChatX backend

MeshChatX uses its self-signed local certificate; clients must use HTTPS and accept the local certificate. The MeshChatX port is not a public ingress and must not be published through Podman, nginx, Cloudflare, or a router. WebODM and MeshChatX must not share a listener. The desktop launcher and watchdog must use port `18000`; changing one without the others is a configuration error.



The current host uses two separate raw-LoRa/Reticulum interfaces:

Interface	Hardware	Frequency	Bandwidth	SF	CR	TX power	Mode
PEOPLE-RADIO	Heltec LoRa 32 V3, SX1262, RNode firmware 1.85	915 MHz	125 kHz	7	5	17 dBm	<code>selected_interface_mode = 1</code>
DRONE-RADIO	Heltec LoRa 32 V3, SX1262, RNode firmware 1.85	917 MHz	250 kHz	7	5	17 dBm	<code>mode = internal;</code> <code>selected_interface_mode = 7;</code> <code>discoverable = no</code>

The different frequencies and airtimes intentionally separate the public people-facing radio from the private drone/ IoT radio. They must not be treated as interchangeable interfaces or combined into one RF channel without an approved frequency plan.

The live host configuration is under `/home/scottw/.reticulum/`. MeshChatX runs headlessly at `127.0.0.1:18000`. Its embedded Reticulum runtime is initialized from `/home/scottw/.reticulum/config`, while MeshChatX identity, repository, and application state are stored under `/home/scottw/.reticulum-meshchatx/`.

Both RNodes are functional and initialize successfully in the active MeshChatX process. This confirms local device detection, serial access, and RNode configuration. RF feedback is observable on both configured bands:

Radio	Configured band	Operational state	Remaining observation
PEOPLE-RADIO	915 MHz	Functional	Characterize feedback observed on this band
DRONE-RADIO	917 MHz	Functional	Characterize feedback observed on this band

“Feedback” is an operator observation, not yet a diagnosed fault. Potential categories include self-feedback, nearby RF activity, interference, harmonics, spurious transmission, antenna coupling, or reflected energy. Do not change power, frequency, bandwidth, spreading factor, coding rate, antenna, or transmit mode until the source and severity are measured.

Both CP2102 bridges expose the same USB serial descriptor `Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001`. Device identity must therefore be resolved through the stable PCI/USB `by-path` location and the recorded SX1262 MAC address. The USB serial descriptor alone is not a unique radio identity.

9.3 Reticulum Interface Inventory

As reviewed on 2026-09-24, `/home/scottw/.reticulum/config` contained 32 configured interfaces:

- Modified: `2026-09-24 10:43:59-07:00`.
- Size: `7942` bytes.
- SHA-256: `2df6a8d9fc2037d9e316ac910ec1721c3b5b6af5e301a50b0e92656226cc4098`.
- 29 enabled `TCPClientInterface` connections.
- Two enabled `RNodeInterface` entries: `PEOPLE-RADIO` and `DRONE-RADIO`.
- One enabled `BackboneInterface` named `Public Gateway`.
- No interface explicitly labeled or configured as disabled.

The configured TCP client interfaces are:

- DX.PE Los Angeles Backbone
- ViscousBits
- Panic Public PDX
- KetronKlassik Oakland Gateway
- hownotbrowncrow
- CenTex1
- jsreed5.org Reticulum Entrypoint
- Unbound Hive TCP
- NecroNet RNS Gateway
- fools-gold-gateway
- CDQ-Drummondville
- Washmesh Gateway
- RazzTech Gateway
- Minuteman RNS West
- rns.amilia.zip
- Montgomery
- GhostMesh ATL IPv6
- CCLLC-Net Public TCP Gateway
- CORE - Central Ohio Radio Enthusiasts
- Mitigomish Public Gateway
- MattInTech Backbone
- rns.quacksradio.com
- Red Nova Gateway
- SCR
- GhostMesh WTX IPv4IPv6
- Parallel RNS-1 Public Hub
- thefossrant us-east-1
- topkeksec
- Simply Equipped US Cloud

“Enabled” records configuration intent only. It does not prove that an interface is connected, reachable, or carrying traffic. Current startup evidence includes successful peering and announces as well as timeouts, unreachable-network errors, connection refusals, and reconnect loops. Historical radio logs also contain `Could not detect device`, `Radio state mismatch`, and retry cycles; the current process later initialized both radios, so those historical events must not be reported as the current state without a fresh capture. Discovered interfaces also appear at runtime and are not a one-to-one match with the static configuration.

The configuration fingerprint identifies only the reviewed static inventory. It does not identify the runtime state of discovered interfaces.

Future interface reports must distinguish:

Field	Required meaning
Interface name	Stable configured or discovered name
Type	TCP client, RNode, backbone, or other transport
Enabled	Configuration intent
Initialized	Device detection and interface configuration completed
Connected	Current socket/interface state
Operational	Recent traffic with no fatal state
Reachable	Successful traffic or peer exchange
Last error	Most recent failure, if any
Last checked	Timestamp of evidence

9.4 Operational Security

The MeshChatX web interface is restricted to `127.0.0.1:18000`.

The Reticulum `Public Gateway` is configured to listen on `0.0.0.0:4242`. Because this listener binds all local IPv4 interfaces, it is not loopback-restricted. Whether it is reachable from the LAN or Internet depends on firewall and upstream controls, which could not be verified without root privileges during the review. The host had `192.168.87.135/24` on `wlp3s0`, making `192.168.87.135:4242` a potential LAN path unless blocked. This does not prove successful external access. A loopback-only web UI does not make the underlying Reticulum gateway private.

9.5 Radio Profile Requirements

Both radio ends must be verified as compatible US915 hardware variants. Matching SX1262-family radio chips do not guarantee protocol compatibility.

Version-controlled profiles:

```
/ALWAYSON/config/field/heltec-v3/radio-profile-us915.yaml
/ALWAYSON/config/drone/waveshare-lora/radio-profile-us915.yaml
```

The profiles must define identical or explicitly interoperable values for:

- Frequency or channel plan.
- Bandwidth.
- Spreading factor.
- Coding rate.
- Preamble length.
- Transmit power.
- Sync word or network identifier.
- Packet framing.
- Maximum packet size.
- Encryption key identifier.
- Device public identity.
- Sequence number and replay-protection policy.
- Acknowledgement policy.
- Retry and backoff policy.
- Airtime limits.

Do not describe this system as LoRaWAN unless it implements a true LoRaWAN device, gateway, and network-server architecture. The field implementation is primarily an RNode-based Reticulum mesh. The Heltec V3 and Raspberry Pi/Waveshare radios must use matched, approved US915 channel plans. Any separate LoRaWAN or public-discussion service must use different radio bands and settings and remain isolated from the field telemetry mesh.

10. Simulation Architecture

10.1 Vehicle Simulation

```
ao-sim-vehicle
├── ROS 2 Lyrical
├── Gazebo Sim 10.5.0
├── ArduPilot SITL
├── ROS-Gazebo bridge
├── MAVLink router
├── QGroundControl simulation client
├── Optional Stable-Baselines3 evaluation
├── Mission and scenario runner
└── Vehicle-result exporter
```

```
ROS_DOMAIN_ID=21
GZ_PARTITION=alwayson_vehicle_sim
```

Vehicle profiles MUST ALLOW FOR SWITCHING BETWEEN THE FOLLOWING, IDEALLY AT RAPID SPEED:

- bicopter profile.
- Fixed-wing VTOL tailsitter profile.
- Rover (QUADCYCLE) profile.
- Dual-rotating underwater/submersible profile.
- BOAT MAST/SAIL CONTROL PROFILE
- BOAT BOW/STERN THRUSTER PROFILE
- BOAT BOW/STERN AIRBOAT FAN PROFILE
- Wind, terrain, obstacles, routing, takeoff, and landing EVENTS
- Camera, GPS, IMU, barometer, rangefinder, battery, and MAVLink behavior.
- GPS loss, packet loss, actuator faults, sensor drift, and failsafe handling.

Vehicle simulation must never connect to live flight controllers, field radios, real drone telemetry, payment services, customer records, or Corda core.

10.2 Fabrication and Facility Simulation

Gazebo Sim model views of the fabrication and facility domain. These show the modelled robot-arm cells, vehicle and shelving layout, and kitchen/storage volumes referenced below. They are **rendered model views, not operational evidence** — no flight-control or live-machinery path is enabled by anything shown here.

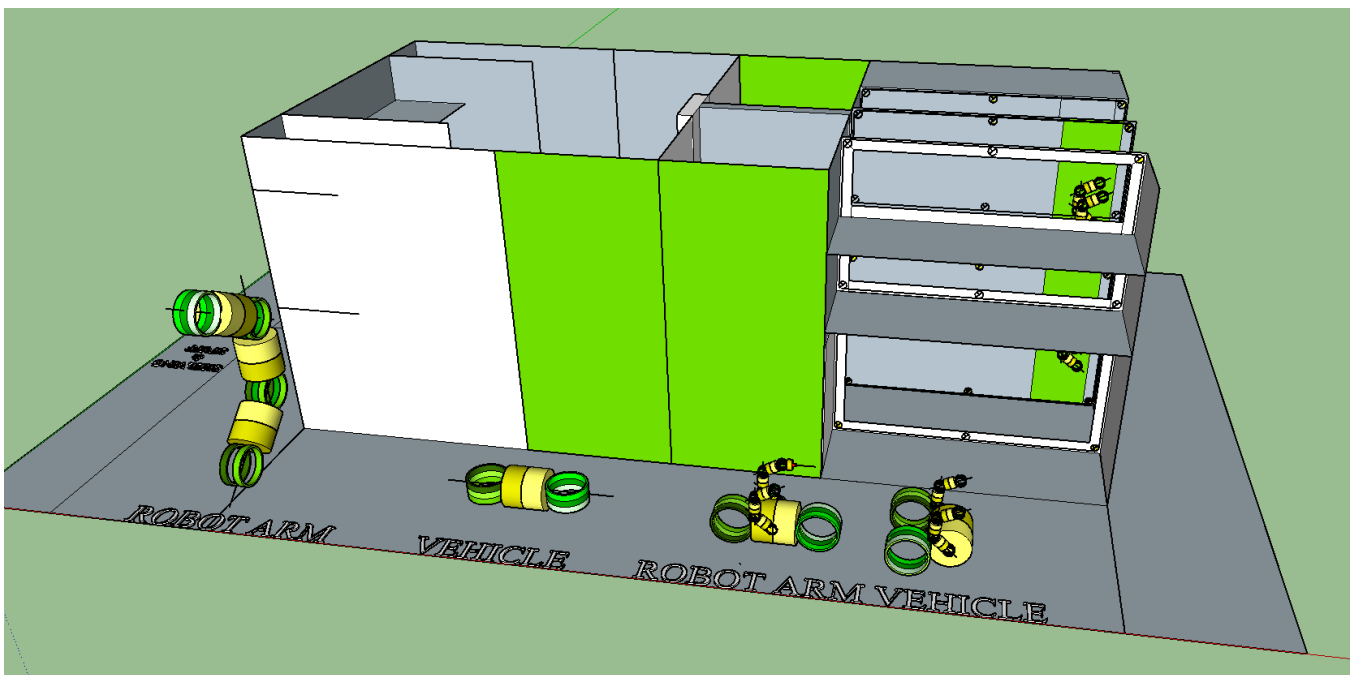


Figure 10.2a — Robot-arm work area, vehicle bay, and robot-arm vehicle bay within the fabrication domain.

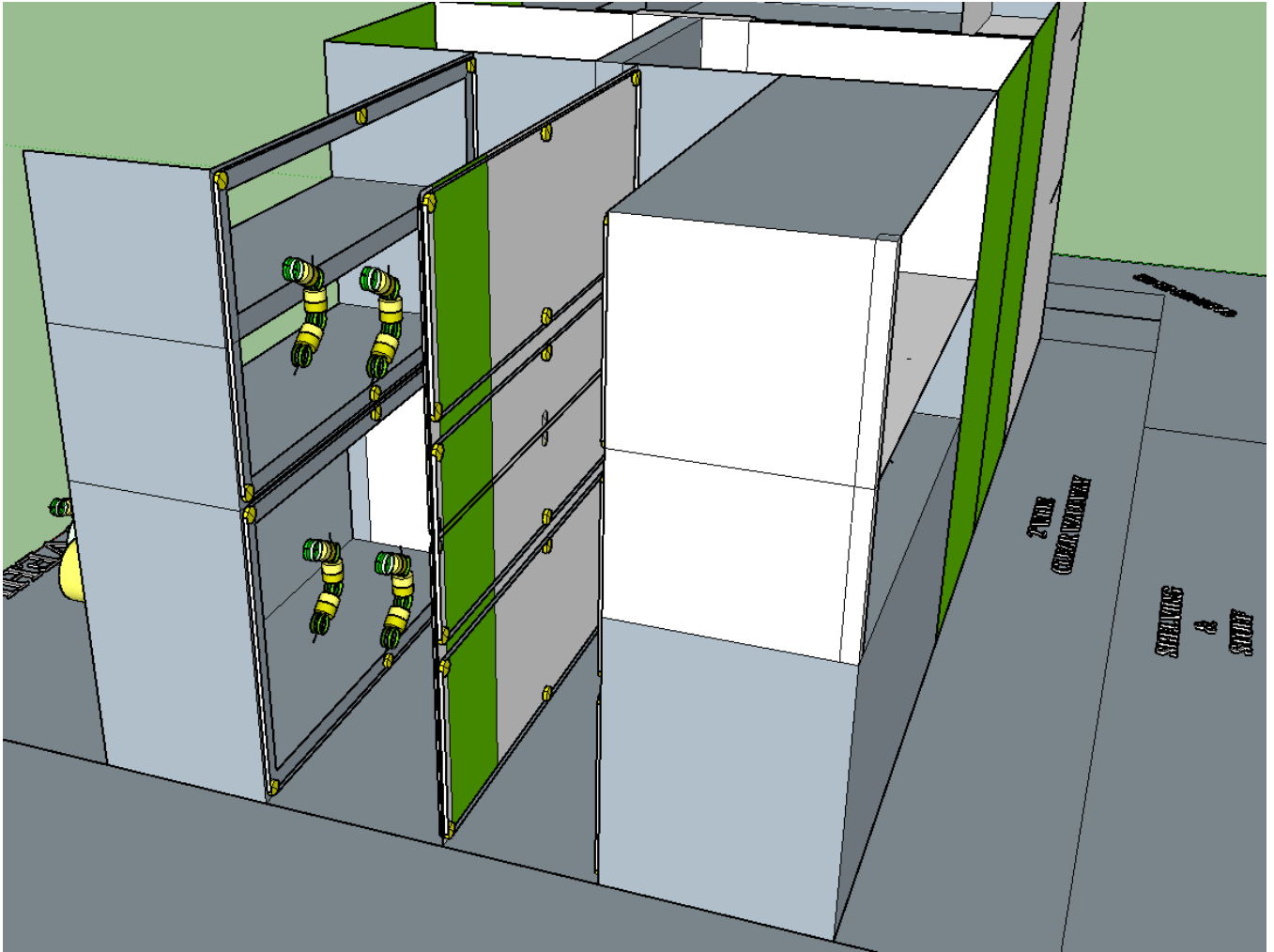


Figure 10.2b — Storage and shelving elevation with robot-arm cells, and the shelving-to-printer aisle.

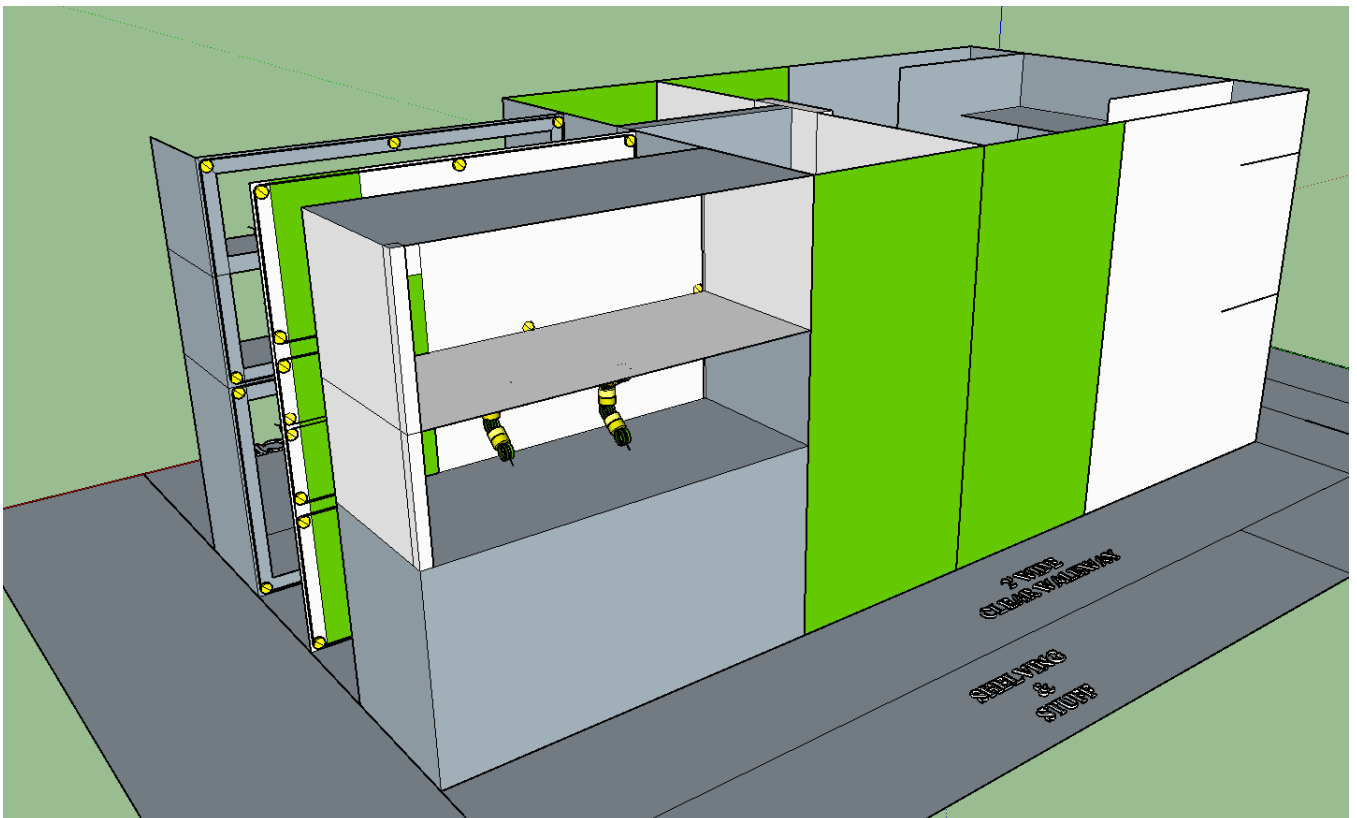
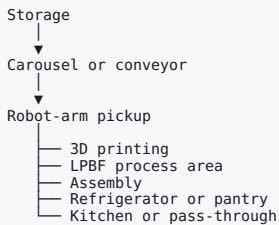


Figure 10.2c — Storage, shelving-to-printer, and kitchen volumes from the reverse approach.

```
ao-sim-fabrication
├── ROS 2 Lyrical
├── Gazebo Sim 10.5.0
├── Robot-arm cells and assembly stations
├── 3D-printer cells
├── LPBF cells
├── Storage and inventory cells
├── Refrigerator, freezer, and pantry models
├── Kitchen and pass-through models
├── Carousels and conveyors
├── Facility scheduler
├── Safety-zone and interlock model
└── Fabrication-result exporter
```

```
ROS_DOMAIN_ID=22
GZ_PARTITION=alwayson_fabrication_sim
```



Phase one is simulation only. It must not command live robot arms, printers, LPBF systems, refrigeration, carousels, kitchen equipment, or other machinery.

Vehicle and fabrication simulation domains require separate:

- Podman networks.
- ROS domain IDs.
- Gazebo partitions.
- DDS configuration.
- Service identities.
- Filesystem mounts.
- Result directories.
- Simulation **manifest-signing** certificates only. These sign exported artifacts; they must not be Corda client identities, and simulation must never open a connection to Corda core (section 10.1).
- Git repositories or clearly separated repository subtrees.
- Artifact manifests.

Use LOCAL FOLDER STORAGE AT /ALWAYSON AT THE GAZEBO SUBFOLDER (VERIFY ITS LOCATION WITH THE USER) for SDF, URDF/Xacro, world files, robot definitions, safety zones, task plans, and launch configurations. Use Git LFS or a separate artifact repository for large meshes, textures, point clouds, and generated results.

11. Ledger, Provenance, Archive, and IPFS

11.1 Ledger Authority Policy

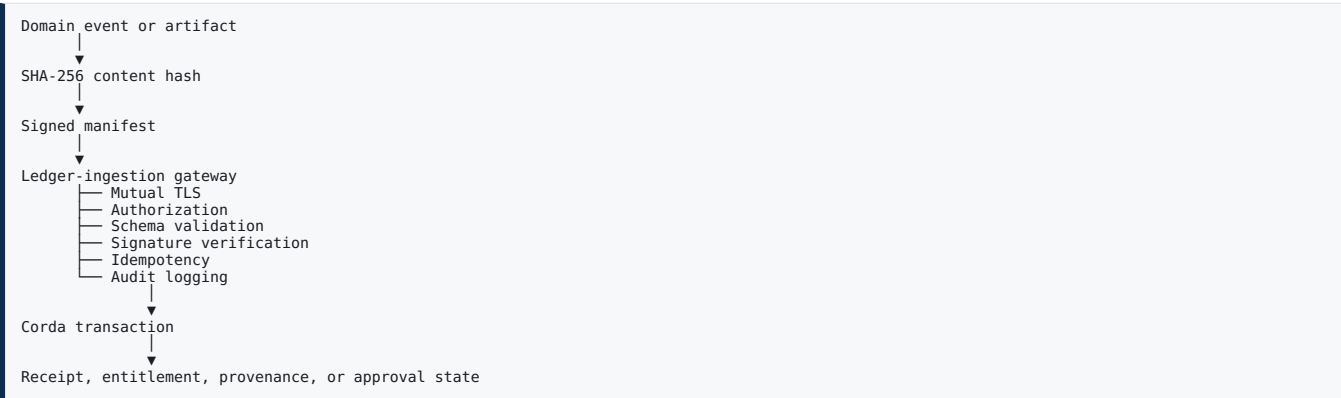
Corda is the authoritative ledger for approved business provenance, receipt, entitlement, fulfillment-approval, and release-approval records.

Corda is not the authoritative store for domain-operational source data (THAT DATA IS TO BE STORED IN THE RELATED POSTGRES SQL DATABASE.)

Domain	Authoritative operational data
Sales	Sales PostgreSQL order, fulfillment, and customer-service records
Payment	Verified provider event record and normalized payment state
Field	Raw packet store, telemetry spool, and mission records
Mapping	Validated imagery, WebODM project data, processing outputs, and deliverables
Vehicle simulation	Scenario definitions, run data, and result artifacts
Fabrication simulation	Facility/task models, safety scenarios, and result artifacts
Archive	Encrypted archive objects and retention records

Corda records signed references, hashes, approved transitions, and entitlement/provenance data that permit verification without duplicating sensitive or high-volume data.

11.2 Ledger Flow



11.2.1 Cross-System Correlation and Provenance Model

The primary business correlation tuple is:

```
serial_number + receipt_number + event_timestamp_utc
```

These fields link records across PostgreSQL domains and approved ledger records:

- `serial_number` identifies the physical product, vehicle, component, or asset.
- `receipt_number` identifies the approved commercial transaction or receipt.
- `event_timestamp_utc` identifies when the source event occurred, using ISO-8601 UTC.

The tuple should be accompanied by a source event identifier and schema version:

```
correlation_id
serial_number
receipt_number
event_timestamp_utc
event_type
source_domain
source_record_id
schema_version
content_hash_sha256
```

Example:

```
{
  "correlation_id": "ORDER-2026-000123-LOT-A",
  "serial_number": "SN-300X3-000042",
  "receipt_number": "RCPT-2026-000123",
  "event_timestamp_utc": "2026-09-25T12:34:56.000Z",
  "event_type": "entitlement_issued",
  "source_domain": "sales",
  "source_record_id": "order-line-000123-01",
  "schema_version": "1.0",
  "content_hash_sha256": "SHA256_DIGEST"
}
```

The same correlation fields should be carried into approved records from sales, mapping, field, fulfillment, simulation, and release workflows where the event is relevant. A database view or reporting projection should join the records by the correlation tuple rather than by free-text names or presentation labels.

Ledger responsibility: integrity and provenance, not general encryption

Corda/blockchain records should contain the minimum data needed to verify that an approved event, artifact, receipt, entitlement, or state transition occurred:

```
serial_number
receipt_number
event_timestamp_utc
event_type
state or status
content_hash_sha256
opaque source reference
signature/authorization metadata
```

The ledger should not contain:

```
card numbers, CVV, payment secrets, private keys,
full customer PII, raw telemetry, imagery, point clouds,
or large operational payloads
```

Encryption is performed before sensitive data leaves its authoritative store, for example before pCloud archival replication or private IPFS distribution. Corda then records the encrypted-object reference and content hash. This gives integrity and provenance for the encrypted object without putting the plaintext payload on the ledger.

Sales and marketing reporting flow

```
Sales PostgreSQL salesdb
├── order, product, serial, receipt, payment, fulfillment, entitlement
│   ├── correlation tuple:
│   │   ├── serial_number + receipt_number + event_timestamp_utc
│   │   └── approved read-only reporting views
│   │       └── Metabase sales/marketing reports
│   ├── optional Grafana PostgreSQL datasource
│   │   └── sales/fulfillment/provenance dashboards (visualization only)
│   └── signed minimized manifest
│       └── Corda receipt/entitlement/provenance state
```

Marketing and sales reporting should use approved PostgreSQL views or projections. The reporting layer may join:

```
product/SKU
serial number
receipt number
order and order-line state
entitlement state
Corda receipt/provenance status
event timestamp
```

It must not infer that a product is fulfilled, entitled, paid, or blockchain-verified solely from a marketing label. Those states must come from the authoritative PostgreSQL event and the approved ledger projection.

Implementation preconditions

Before enabling this flow:

1. Initialize and verify the `salesdb` schema.
2. Define canonical `serial_number`, `receipt_number`, and UTC timestamp fields.
3. Create read-only reporting views for Metabase, and for Grafana where enabled.
4. Define the signed manifest schema and correlation-ID uniqueness rule.
5. Implement ledger-ingest authorization, signature verification, idempotency, replay protection, and audit logging.
6. Complete the Corda key/certificate ceremony.
7. Test the complete correlation path with synthetic data before connecting real sales, payment, customer, or product records.

11.2.2 Mandatory Corda Entry Evidence

Corda entry is blocked until all three evidence classes are present for the same business correlation record:

1. **Sale-request email** - Customer-originated sale/KIT REQUEST email or approved equivalent. - Captures requester, requested items/SKUs, comments, and request timestamp. - Does not by itself prove a contract or payment.
2. **Payment-validation email** - Provider-specific validation for PayPal, Zelle, or Coinbase/stablecoin. - Identifies the provider, provider reference, amount, currency, validation status, and validation timestamp. - Does not by itself prove that funds settled into the approved account.
3. **Funds-transfer verification** - Operator/provider reconciliation evidence that the funds actually transferred and settled. - Records settlement/available state, transfer reference, amount, currency, and verification timestamp. - Must not be treated as verified merely because a payment was initiated.

The three records must resolve to the same:

```
correlation_id
receipt_number
serial_number(s)
event_timestamp_utc
```

Recommended evidence record:

```
evidence_id
evidence_type = sale_request | payment_validation | funds_transfer_verification
provider = website | paypal | zelle | coinbase | bank | manual_reconciliation
source_reference
received_at_utc
validated_by
content_hash_sha256
status = received | validated | rejected | superseded
```

A sale is not eligible for Corda submission unless:

```
sale_request.status = validated
payment_validation.status = validated
funds_transfer_verification.status = validated
```

Corda records references, hashes, states, and operator authorization for these three gates; it must not store raw payment credentials or unrestricted email content. PostgreSQL stores the detailed evidence metadata and reporting projection. Metabase and Grafana report the resulting confirmed state; they do not perform or waive the verification.

1. Complete the Corda key/certificate ceremony.
2. Test the complete correlation path with synthetic data before connecting real sales, payment, customer, or product records.

11.2.3 Corda-Managed Sale and Receipt Process

The detailed process, state machine, correlation model, and activation gate are maintained in the canonical runbook:

```
/ALWAYSON/docs/runbooks/corda-sale-receipt-process.md
```

The short rule is:

```
KIT REQUEST/inquiry
→ verified payment
→ PostgreSQL provisional projection
→ signed sale-contract manifest
→ ledger-ingest gateway
→ Corda transaction/state
→ PostgreSQL final projection
→ receipt and reporting views
```

A receipt is not final until Corda has returned a confirmed transaction/state reference and the PostgreSQL projection records that reference. The intake, form, schema, and validator artifacts are:

```
/ALWAYSON/data/sales/kit-request-intake/
/ALWAYSON/forms/three-column-corda-sale-receipt-form.html
/ALWAYSON/forms/three-column-corda-sale-receipt-form.pdf
/ALWAYSON/forms/corda-sale-receipt.html
/ALWAYSON/config/sales/sale-receipt.schema.json
/ALWAYSON/config/sales/sale-receipt.example.json
/ALWAYSON/scripts/validation/validate-sale-receipt.sh
```

The form is an internal operator form. It does not write to PostgreSQL, contact Corda, process payments, or create a public proof. Submission must go through the authorized ledger-ingest workflow after operator review.

11.3 Corda Stores and Private Data

Corda may retain approved private transaction data as an encrypted private payload or encrypted attachment. Corda does not make plaintext private data safe merely by being on a ledger: confidentiality depends on encryption, authorized recipients, key management, access policy, and audit controls.

Corda contract state

Corda state should contain the small, shared, verifiable business facts:

```
transaction_id
correlation_id
receipt_number
order_id
serial_number(s)
sku
model object id / model_revision_id
payment_provider
payment-validation reference/hash
funds-transfer reference/hash
payment/settlement state
entitlement/fulfillment/delivery state
Corda transaction ID
timestamps
signatures/authorization metadata
```

Encrypted private payload

Approved private data may be encrypted before submission and stored as a private attachment or confidential private-state object:

```
customer identity and contact details
purchase-request email/content
payment-validation email/content
funds-transfer verification content
full receipt and contract
fulfillment, delivery, return, and support records
private 3D model files and attachments
```

The private payload envelope must include:

```
transaction_id
data_classification = PRIVATE
schema_version
encryption_algorithm
encryption_key_identifier
authorized_recipients
payload_SHA-256
retention_policy_identifier
created_at_utc
```

The encryption key must be held by the approved KMS/wallet/key-management process and must never be stored in Corda, PostgreSQL, Git, HTML, logs, or the transaction bundle.

Never store in Corda

```
card numbers
CVV
bank credentials
payment-provider secret keys
passwords
OAuth tokens
private keys
TLS private keys
KMS master keys
data-encryption keys
recovery phrases
```

Corda tracks the transaction ID, state, hashes, references, and authorized signatures. PostgreSQL retains the operational/reporting projection keyed by the same transaction ID. Metabase and Grafana report the confirmed state; they do not create or waive payment verification.

11.4 Corda Does Not Store

Corda may store approved encrypted private transaction data as described in Section 11.3. It must never store plaintext secrets or unencrypted credentials.

Never store:

```
card numbers
CVV
bank credentials
payment-provider secret keys
passwords
OAuth tokens
private keys
TLS private keys
KMS master keys
data-encryption keys
recovery phrases
```

11.5 Manifest Format

```
{
  "object_id": "UUID",
  "object_type": "sales_receipt | telemetry_batch | map_product | vehicle_simulation | fabrication_simulation",
  "origin_domain": "sales | field | mapping | sim_vehicle | sim_fabrication",
  "created_at_utc": "ISO-8601 UTC timestamp",
  "schema_version": "1.0",
  "content_hash_sha256": "HEX_DIGEST",
  "content_size_bytes": 0,
  "local_storage_reference": "opaque internal reference",
  "ipfs_cid": "optional encrypted CID",
  "pcloud_archive_reference": "optional opaque encrypted reference",
  "authorization_policy_id": "policy ID",
  "producer_key_id": "service key ID",
  "signature": "detached signature"
}
```

11.6 pCloud and IPFS Rules

```
Local source data
├── Content hash
├── Signed manifest
├── Corda receipt or approval state
├── Encrypted pCloud archive
└── Private or encrypted IPFS distribution
```

- Local source data remains authoritative.
- Encrypt before pCloud archival replication unless an explicitly approved equivalent encryption control applies.
- Do not place private data, PII, payment data, private keys, raw telemetry, sensitive imagery, or proprietary technical designs on public IPFS.
- For post-sale transfer of sensitive artifacts, use a private IPFS swarm, controlled pinning, or encryption before any public IPFS distribution.
- Record content hash and CID separately.
- Post-sale, store only CIDs and encrypted package references in Corda. IPFS is a post-sale marking/transfer mechanism only (ES.1); it is not archive, accounting, or ledger storage, and Corda has no IPFS dependency.
- Archive replication occurs only through [ao-egress-archive](#).

12. Host Installation and Configuration

12.1 Installation Journal

Create the journal before installation activity:

```
sudo install -d -m 0750 -o "$USER" -g "$USER" /ALWAYSON/logs/installation
touch /ALWAYSON/logs/installation/agent-install.log
chmod 0640 /ALWAYSON/logs/installation/agent-install.log
```

12.2 Initial Non-Destructive Inventory

Run before installing or changing anything:

```
{
  echo "===== Timestamp ====="
  date --iso-8601=seconds

  echo "===== Host ====="
  hostnamectl

  echo "===== OS ====="
  cat /etc/os-release

  echo "===== Kernel ====="
  uname -a

  echo "===== CPU / RAM ====="
  lscpu
  free -h

  echo "===== Storage ====="
  lsblk -o NAME,SIZE,FSTYPE,FSVER,LABEL,UUID,MOUNTPOINTS
  df -hT

  echo "===== Photogrammetry Mount ====="
  findmnt /media/scottw/500GBPHOTOGRAM || true

  echo "===== Podman ====="
  command -v podman || true
  podman version 2>&1 || true
  podman info 2>&1 || true

  echo "===== systemd ====="
  systemd --version

  echo "===== cgroups ====="
  stat -fc %T /sys/fs/cgroup

  echo "===== GPU ====="
  lspci -nnk | grep -A3 -Ei 'VGA|3D|NVIDIA' || true
  command -v nvidia-smi && nvidia-smi || true

  echo "===== Network ====="
  ip -brief address
  ss -tulpn
  ss -tulpn6

  echo "===== Firewall ====="
  sudo ufw status verbose 2>&1 || true
  sudo nft list ruleset 2>&1 || true

  echo "===== Existing systemd services ====="
  systemctl --user list-unit-files --type=service 2>&1 || true

  echo "===== Existing containers: current user ====="
  podman ps -a 2>&1 || true

  echo "===== Existing containers: system store ====="
  sudo podman ps -a 2>&1 || true

  echo "===== Existing Podman networks: current user ====="
  podman network ls 2>&1 || true

  echo "===== Existing Podman networks: system store ====="
  sudo podman network ls 2>&1 || true

  echo "===== Reticulum and MeshChatX ====="
  command -v rnsd || true
  rnsd --version 2>&1 || true
  pgrep -a -f 'rnsd|ReticulumMeshChatX' || true
  ss -ltnp 2>/dev/null | grep -E '(:18000|:4242)' || true

  if [ -f "$HOME/.reticulum/config" ]; then
    echo "--- Reticulum configuration ---"
    grep -nE '^\\[|^type =|^interface_enabled|enabled) =|^target_host =|^target_port =|^port =|^mode =' \
      "$HOME/.reticulum/config"
  fi

  if [ -d "$HOME/.reticulum-meshchatx/logs" ]; then
    echo "--- Recent MeshChatX errors and warnings ---"
    tail -n 500 "$HOME/.reticulum-meshchatx/logs/meshchatx.log" \
      | grep -Ei 'error|warning|disabled|interface|umsgpack' || true
  fi

  echo "===== Serial devices ====="
  ls -l /dev/serial/by-id/ 2>&1 || true
} | tee -a /ALWAYSON/logs/installation/agent-install.log
```

Pause and report if:

- The photogrammetry drive is not mounted.

- The mountpoint is an ordinary root-filesystem directory.
- Mapping storage is below 100 GB free.
- Existing WebODM, Podman, Docker, Corda, PostgreSQL, ROS, Gazebo, or related services conflict.
- NVIDIA driver state is broken.
- cgroups v2 or the selected Podman runtime mode does not work.
- Firewall policy conflicts with intended isolation.
- A proposed service port is already bound.
- Heltec cannot be found through a stable `/dev/serial/by-id/` path.
- MeshChatX or Reticulum is unexpectedly absent after installation approval.
- The Reticulum configuration contains an interface not present in the approved inventory.
- A supposedly enabled interface repeatedly fails without a documented compensating control.
- The Reticulum gateway listener is reachable from an unapproved network.
- MeshChatX reports persistence, cryptographic-state, or repository-integrity errors.

12.3 Host Dependencies

After inventory review and explicit operator approval:

```
sudo apt update

sudo apt install -y \
  podman \
  uidmap \
  slirp4netns \
  fuse-overlayfs \
  containernetworking-plugins \
  nftables \
  ufw \
  git \
  curl \
  jq \
  ca-certificates \
  gnupg \
  openssl \
  restic \
  smartmontools \
  lm-sensors \
  acl \
  python3 \
  python3-venv \
  python3-pip
```

Verify:

```
podman version
podman info --debug
systemctl --user status
loginctl show-user "$USER" -p Linger
test "$(stat -fc %T /sys/fs/cgroup)" = "cgroup2fs" && echo "cgroups v2 active"
sudo aa-status || true
```

13. Podman Runtime and Quadlet Policy

13.1 Rootless and System-Level Podman

Ordinary application workloads should use rootless Podman and user-level Quadlet units.

Rootless Quadlet definitions are normally stored in:

```
~/.config/containers/systemd/
```

System-level Quadlet definitions are stored in:

```
/etc/containers/systemd/
```

System-level services are permitted only where a documented host-hardware, GPU, storage, networking, or service-management requirement makes rootless operation unsuitable.

13.2 Approved Deviation: Mixed Podman Stores

Architecture requirement: Rootless Podman is the preferred default for ordinary workloads.

Current implementation: Mapping smoke-test evidence indicates that at least some WebODM operations executed through the system/rootful Podman store. The evidence includes root-owned mapping backup artifacts and system-side container storage. An empty `podman ps -a` result from an operator shell does not mean system-store containers, images, volumes, or networks are absent.

Compensating controls:

- No `--privileged` containers.
- Internal mapping network only.
- Explicit bind mounts limited to approved mapping paths.
- Pinned image digests.
- systemd resource limits and restart policy.
- Validated NVIDIA CDI access only where required.
- No direct public listener.
- Backup and restore evidence retained.

Resolution condition: Before production declaration, record the approved steady-state model for every domain: rootless, system-level, or mixed. Record the unit owner, Quadlet location, storage path, network owner, GPU access method, and rationale.

13.3 /ALWAYSON Layout

```
/ALWAYSON/  
├── README.md  
├── VERSION  
├── docs/  
├── storefront/  
│   ├── source/  
│   ├── build/  
│   ├── releases/  
│   ├── manifests/  
│   └── pcloud-public-folder/  
├── quadlet/  
│   ├── networks/  
│   ├── volumes/  
│   ├── sales/  
│   ├── payment/  
│   ├── field/  
│   ├── mapping/  
│   ├── sim-vehicle/  
│   ├── sim-fabrication/  
│   ├── ledger/  
│   ├── archive/  
│   └── operations/  
├── config/  
│   ├── platform/  
│   ├── storefront/  
│   ├── sales/  
│   ├── payment/  
│   ├── drone/  
│   ├── field/  
│   ├── mapping/  
│   ├── sim-vehicle/  
│   ├── sim-fabrication/  
│   ├── ledger/  
│   ├── mastodon/  
│   ├── pcloud/  
│   └── ipfs/  
├── secrets/  
├── data/  
├── artifacts/  
├── ipfs/  
├── pcloud/  
├── backups/  
├── logs/  
├── scripts/  
├── tests/  
└── tmp/
```

Initialize source control:

```
cd /ALWAYSON  
git init  
git branch -M main  
  
cat > .gitignore <<'EOF'  
secrets/  
data/  
logs/  
tmp/  
backups/  
pcloud/restore-cache/  
EOF  
  
git add .gitignore  
git commit -m "Initialize ALWAYS ON configuration repository"
```

13.4 Quadlet Network Template

```
# /ALWAYSON/quadlet/networks/ao-mapping.network  
[Network]  
NetworkName=ao-mapping  
Driver=bridge  
Internal=true
```

13.5 Quadlet Service Template

```
# /ALWAYSON/quadlet/mapping/webodm-web.container  
[Unit]  
Description=ALWAYS ON WebODM Web Service  
After=network-online.target  
Wants=network-online.target  
  
[Container]  
Image=REPLACE_WITH_APPROVED_IMAGE_DIGEST  
ContainerName=webodm-web  
Network=ao-mapping.network  
Volume=/media/scottw/500GBPHOTOGRAM/webodm/media:/webodm/app/media:Z  
Volume=/ALWAYSON/config/mapping/webodm:/config:ro,Z  
NoNewPrivileges=true  
  
[Service]  
Restart=on-failure  
RestartSec=15  
MemoryMax=12G  
CPUQuota=600%  
TimeoutStartSec=180  
  
[Install]  
WantedBy=default.target
```

This is a structural template only. The exact image, API settings, mounts, service name, environment, and GPU configuration must be taken from the tested and approved WebODM version.

14. Secrets, Service Identity, and Version Controls

14.1 Secret Delivery

Use Podman secrets or systemd credentials. Prefer file-based secret delivery rather than environment variables.

Secret	Authorized domain
Sales database password	Sales only
Payment webhook secret	Payment verifier only
Payment-provider API secret	Payment adapter only
Mastodon OAuth credential	Community adapter only
Local AI credential/configuration if required	AI service only
Field radio key	Field only
Drone signing key	Drone device only
Corda certificates and keystores	Ledger core only
Ledger client certificates	One distinct certificate per exporter/domain
pCloud archive credential	Archive adapter only
IPFS private-swarm/pinning credential	Archive adapter only

Example:

```
[Container]
Secret=sales_db_password,target=/run/secrets/db_password,uid=10001,gid=10001,mode=0400
```

KDE Wallet may hold interactive operator credentials, but unattended production services must use systemd credentials, Podman secrets, or approved service-specific secret files. Secret rotation, revocation, expiration, and recovery procedures must be documented before production use.

14.1.1 KDE Wallet Secret Management (Implemented)

KDE Wallet is the operator-side secret and credential store for this host. This subsection records the implemented integration; the Section 14.1 policy above remains authoritative, and the unattended-delivery deviation is tracked in Section 18 (Open Decisions).

Runtime and tooling:

- Wallet daemon: `kwalletd6`, reached on the `org.kde.kwallet6` D-Bus name. Wallet: `kdewallet`, auto-unlocked with the operator's Plasma login.

Verification pending (operator, host check). The bus name and the `entryList/hasEntry` method names are to be confirmed on the running host before anything depends on them. KDE Wallet remains the secret authority per ES.1 regardless of the outcome; only these access details are unconfirmed. Do not rely on an unverified name in a script — confirm it first, then record the verified values here.

- Management CLI: `scripts/ops/kwallet-provision.sh` (`create-folders`, `put`, `get`). Run only from the interactive Plasma session while the wallet is unlocked.
- Boot-time delivery: `scripts/operations/fetch-kwallet-secret.sh` runs as a Quadlet `ExecStartPre`, waits for the desktop session and `kwalletd` (max ~60s), reads the required entries, and writes a service-specific `0600` env file under the unit owner's `~/secrets/` for the unit to consume via `--env-file`. Used by `ao-mastodon-db`, `ao-sales-db`, and `ao-webodm-db` (verified at boot; see the installation journal).

This login-gated behaviour is intended, not a defect. Services that consume Wallet secrets start after the operator's Plasma login and are not expected to start unattended before a user has entered the password. The ~60s wait is the bounded startup allowance for that login, not a fallback that must survive a passwordless boot. Auto-login, if ever enabled, is a convenience for the operator and is not a requirement of this design.

Single service account. All services run under the operator's own account. No service requires a separate service-account user; the `always-on-sales` (UID 993) Mastodon placement described in section 20.0 is legacy and is not a required or intended arrangement. It is recorded in section 20.0 as observed history and should be consolidated back to the operator account.

Wallet layout (folder: purpose):

Folder	Purpose
ALWAYSON	Boot-time delivery entries consumed by Quadlet units
ao-mastodon	Local 300X3 Mastodon application secrets (Section 15.3) and OpenClaw OAuth material
ao-sales, ao-payment, ao-field, ao-mapping, ao-ledger, ao-archive, ao-admin, ao-sim-vehicle, ao-sim-fabrication	Per-domain credential folders matching the Section 14.1 authorized-domain table (provisioned empty 2026-08-31)

Current entry inventory (names only; values never in Git, logs, or docs):

Folder	Entries
ALWAYSON	mastodon-db-password, sales-db-password, webodm-postgres-password
ao-mastodon	mastodon-secret-key-base, mastodon-otp-secret, mastodon-db-password, mastodon-ar-deterministic-key, mastodon-ar-primary-key, mastodon-ar-derivation-salt, mastodon-admin-password, openclaw-bot-client-id, openclaw-bot-client-secret, openclaw-bot-access-token, openclaw-bot-password, roundtrip/roundtrip2 (test artifacts)

Rules:

- Never print, copy, export, or log entry values; confirm presence only Presence checks use the D-Bus `entryList/hasEntry` methods on `org.kde.kwallet6`.
- Entries are named per service and per purpose; domain folders enforce the Section 14.1 authorized-domain boundaries.
- Rotation, revocation, expiration, and recovery procedures must be documented before production use (Section 14.1 requirement).

14.2 Version Matrix

Maintain:

/ALWAYSON/config/platform/version-matrix.yaml

```
host:
  os_release: ""
  kernel: ""
  systemd: ""
  podman: ""
  quadlet_capability: ""
  netplan: ""
  nftables: ""
  ufw: ""

gpu:
  model: "EVGA NVIDIA GTX 1080"
  nvidia_driver: ""
  container_runtime_integration: ""
  cuda_runtime_image_digest: ""

mapping:
  webodm_image_digest: ""
  nodeodm_image_digest: ""
  postgresql_version: ""
  redis_version: ""
  processing_profiles_commit: ""

simulation:
  ros2_distribution: "lyrical"
  gazebo_release: "10.5.0"
  ardupilot_commit: ""
  qgroundcontrol_version: ""
  sb3_version: ""

field_chat:
  standalone_rnsd_version: "1.4.2"
  standalone_rnsd_executable: "/home/scottw/.local/bin/rnsd"
  standalone_rnsd_running: false
  active_reticulum_runtime: "embedded in MeshChatX native backend"
  active_embedded_rns_version: "unverified"
  reticulum_config: "/home/scottw/.reticulum/config"
  reticulum_config_sha256: "2df6a8d9fc2037d9e316ac910ec1721c3b5b6af5e301a50b0e92656226cc4098"
  meshchatx_launcher_metadata_version: "4.9.1"
  meshchatx_running_version: "unverified"
  meshchatx_executable: "/home/scottw/Applications/meshchatx-native/ReticulumMeshChatX"
  meshchatx_executable_sha256: "4f403e52b0a5722a49d433f23660b14b43a779fb3cc9a5a90b8f150d77f18890"
  meshchatx_manifest_sha256_match: true
  meshchatx_storage: "/home/scottw/.reticulum-meshchatx"
  meshchatx_local_ui: "127.0.0.1:18000"
  reticulum_public_listener: "0.0.0.0:4242"
  repository_cached_artifact: "reticulum_meshchatx-4.8.4-py3-none-any.whl"
  repository_cached_artifact_running: false

people_radio:
  hardware: "Heltec WiFi LoRa 32 V3 / SX1262"
  rnode_firmware: "1.85"
  frequency_hz: 915000000
  bandwidth_hz: 125000
  spreading_factor: 7
  coding_rate: 5
  txpower_dbm: 17

drone_radio:
  hardware: "Heltec WiFi LoRa 32 V3 / SX1262"
  rnode_firmware: "1.85"
  frequency_hz: 917000000
  bandwidth_hz: 250000
  spreading_factor: 7
  coding_rate: 5
  txpower_dbm: 17
  mode: "internal"
  discoverable: false

ledger:
  corda_version: ""
  cordapp_hashes: ""
  postgres_version: ""
  certificate_profile_version: ""
```

15. Sales, Mastodon, OpenClaw, and Local AI

15.1 Sales Database

Use a dedicated sales PostgreSQL database with separate roles:

```
salesdb
sales_api_role
sales_migration_role
sales_backup_role
sales_reporting_role
sales_admin_role
```

The desktop metadata reports MeshChatX 4.9.1. The native executable hash matches `backend-manifest.json`, but its running version was not independently established. A `reticulum_meshchatx-4.8.4-py3-none-any.whl` artifact also exists in the local MeshChatX repository-server identity and is not the verified running artifact.

Core tables:

```
customers
customer_contacts
products
product_versions
orders
order_lines
payment_provider_events
payment_references
receipts
fulfillment_events
entitlements
returns
support_cases
audit_events
```

15.1.1 Three-Form Transaction Bundles

Every purchase transaction uses one ALWAYS ON-issued transaction ID and one folder containing three forms:

```
/ALWAYSON/data/sales/transactions/<transaction-id>/
├── 01-purchase-request.html
├── 02-payment-confirmation.html
├── 03-receipt.html
├── BUNDLE-STATUS.txt
├── provider-evidence/
│   ├── paypal.*
│   ├── zelle.*
│   └── coinbase.*
```

Issue a new bundle:

```
/ALWAYSON/scripts/sales/issue-transaction-bundle.sh
```

The issuer creates a unique ID, pre-fills that ID into all three forms, and creates the provider-evidence directory. The three forms are:

1. Purchase request.
2. Payment confirmation, including provider validation and funds-transfer settlement.
3. Corda receipt, including the three evidence references and Corda state.

Validate the bundle structure:

```
/ALWAYSON/scripts/sales/validate-transaction-bundle.sh \
/ALWAYSON/data/sales/transactions/<transaction-id>
```

The same issued ID must appear in all three forms and in the bundle status. It is also required in the Corda sale-receipt event and the PostgreSQL contract projection. Payment validation must distinguish provider validation from funds settlement, and the receipt must record the three validated evidence references before operator handoff to ledger-ingest.

Corda tracks the transaction ID, payment/ledger state, hashes, and approved references. Detailed private data remains in the encrypted PostgreSQL projection keyed by the same transaction ID; Corda does not store full customer records, raw emails, payment credentials, or unrestricted evidence.

15.1.2 Website KIT REQUEST PDF Intake

Website-generated PDF requests are accepted at:

```
/ALWAYSON/data/sales/kit-request-intake/
```

The current 300x3.com KIT REQUEST flow composes an email with:

```
SUBJECT: 300X3-WEBREQUEST-
NAME: <name>
EMAIL: <email>
KIT REQUESTED: <selected kits>
COMMENTS: <comments>

THIS IS A REQUEST FOR INFORMATION, NOT A CONTRACT
```

The intake folder separates requests from sales:

inbox/	Original PDFs placed for intake
extracted/	Extracted text
manifests/	Hashes and intake metadata
receipts/	Final receipts/contracts only after Corda confirmation
review/	Human review records
archive/	Preserved processed request PDFs
quarantine/	Invalid, duplicate, or sensitive-pattern PDFs

Run the non-destructive intake script:

```
/ALWAYSON/scripts/sales/intake-kit-request-pdf.sh \
/ALWAYSON/data/sales/kit-request-intake/inbox/<request>.pdf
```

The script preserves and hashes the PDF, extracts text, creates a review record, and explicitly sets:

```
classification=kit_request_inquiry
sale_logged=false
corda_state=NOT_SUBMITTED
```

It never treats a website request as payment, creates an order, or submits a Corda transaction. A verified payment event is required before the PostgreSQL sale projection is created. A final receipt requires a Corda-confirmed transaction/state reference written back to PostgreSQL. Metabase and Grafana read the resulting approved projections; they do not create the sale.

15.2 Community and AI Controls

Mastodon/community controls:

- Dedicated OAuth registration.
- Minimum necessary scopes.
- External access only through [ao-egress-community](#) when explicitly enabled.
- Rate limits.
- Separate approval workflow.
- Immutable publication audit log.
- No payment, field, mapping, simulation, or Corda-core access.

OpenClaw uses the local LM Studio model for support drafting and the deployed [mastodon-openclaw-bridge.service](#) automatically answers new Mastodon mentions and replies as [bot](#). The bridge polls the local Mastodon API every 10 seconds, persists its notification cursor, skips historical notifications and its own posts, and posts threaded public replies locally. It does not publish to any other service. Human approval remains required for pricing, orders, shipping, warranties, financial topics, technical claims, safety guidance, legal statements, and any publication outside the local bridge workflow.

15.3 Local 300X3 Mastodon Deployment

The 300X3 Mastodon instance (Mastodon 4.3.7, containerized in the authoritative [alwayson-sales](#) rootless Podman store) is publicly federated at <https://mastodon.300x3.com>. The main storefront remains on <https://300x3.com> and <https://www.300x3.com>; it is not routed to Mastodon. Operators use Konqueror and OpenClaw on the desktop. Tokodon is removed completely (ES.1) and is not a supported client.

Architecture requirements and verified state:

- [ao-sales](#) remains `Internal=true` and contains the Mastodon database, Redis, streaming service, and web origin. Database and Redis are not attached to the egress network.
- [ao-egress-community](#) is a separate non-internal bridge attached only to [mastodon-web](#) and [mastodon-sidekiq](#). It provides controlled outbound federation delivery; no database, Redis, or streaming container is attached.
- Origin web and streaming remain loopback-only: `127.0.0.1:3000` and `127.0.0.1:4000`.

- The sole public Mastodon entry is the dedicated Cloudflare Tunnel hostname `mastodon.300x3.com`, routed to `127.0.0.1:3000` by `cloudflared-alwayson.service`. The storefront hostnames are excluded from the Mastodon tunnel and retain the filedn redirect behavior.
- Mastodon identity is `LOCAL_DOMAIN=mastodon.300x3.com`. The local user records retain login emails `admin@300x3.com` and `bot@300x3.com`, while their canonical ActivityPub identities are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.
- Public actor and WebFinger endpoints were verified at `https://mastodon.300x3.com/actor` and `https://mastodon.300x3.com/.well-known/webfinger`.
- The tunnel currently uses HTTP/2 transport because QUIC stream timeouts were observed on this host. Local and public health checks returned HTTP 200.
- Open registration remains enabled with the approval gate; approval applies to new account registration, not to following an existing local account.
- No passwords, OAuth secrets, API keys, tunnel credentials, or access tokens are committed to Git or recorded in this README.

15.4 Federation Publication of the Local 300X3 Instance

Category: Implemented and operational (completed 2026-09-24).

Scope. Join the fediverse as the 300X3 instance so that public posts from the local deployment appear on external Mastodon servers, including `mastodon.social`.

Federation is a mutual, inbound-and-outbound protocol: remote servers (including `mastodon.social`) must reach this instance over the public internet using HTTPS, and this instance must be able to deliver outbound activity to remote inboxes. The former loopback-only validation stage (Section 15.3) is superseded by the Cloudflare Tunnel edge. Federation is publicly reachable at `https://mastodon.300x3.com`; the main storefront remains on the apex/`www` hostnames and is not routed to Mastodon.

15.4.1 Architecture Requirements

Identity is verified against the live instance: WebFinger and `/api/v1/instance` both report `mastodon.300x3.com`, while `300x3.com` serves the static storefront. The `scottw` and `alwayson-sales` service accounts are separated so the desktop user cannot start a second Mastodon (Section 20.0). Open configuration drift against these values is tracked in section 19.3.

Area	Architecture requirement
Public instance domain	Dedicated <code>mastodon.300x3.com</code> ; canonical handles are <code>user@mastodon.300x3.com</code> . The storefront hostnames remain separate.
Storefront preservation	<code>300x3.com</code> and <code>www.300x3.com</code> retain the filedn static-site redirect; Mastodon is not deployed under a <code>/mastodon</code> subpath.
TLS	Required at the public edge; Cloudflare terminates TLS for <code>mastodon.300x3.com</code> .
Inbound reachability	Cloudflare Tunnel connector <code>cloudflared-alwayson.service</code> routes only the dedicated hostname to <code>127.0.0.1:3000</code> .
Outbound reachability	<code>mastodon-web</code> and <code>mastodon-sideiq</code> use <code>ao-egress-community</code> for federation delivery; database, Redis, and streaming remain isolated on <code>ao-sales</code> .
Isolation	<code>ao-sales</code> remains <code>Internal=true</code> ; no database, Redis, or raw origin listener is publicly exposed.
Secrets	Tunnel credentials and API keys remain in protected runtime secret storage; never in Git or this README.
Operator duties	Registration approval, moderation, reports, and blocklists remain operator responsibilities.
Service-account placement	The 5 Mastodon containers run under <code>alwayson-sales</code> (UID 993) in a separate rootless store and systemd user manager , not under <code>scottw</code> . <code>ao-mastodon-web.service</code> / <code>ao-mastodon-streaming.service</code> are masked in the <code>scottw</code> manager to prevent a duplicate instance.

15.4.2 Domain and Mastodon Identity Configuration

Environment changes applied to the authoritative service-account `mastodon.env` on 2026-09-24:

```
LOCAL_DOMAIN=mastodon.300x3.com
LOCAL_HTTPS=true
RAILS_FORCE_SSL=false # Cloudflare edge terminates public TLS
ALTERNATE_DOMAINS=localhost,127.0.0.1
```

- Login emails remain `admin@300x3.com` and `bot@300x3.com`.
- Canonical ActivityPub identities are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.

- WebFinger and actor JSON were verified through the public federation hostname.
- The main storefront remains on `300x3.com` / `www.300x3.com`.
- No `/mastodon` path deployment is used; the dedicated hostname provides the root paths required by ActivityPub.

15.4.3 Edge, TLS, and Network Path

Implemented path (2026-09-22, operator-approved Cloudflare Tunnel variant; workstation-nginx + Let's Encrypt variant below superseded — see Section 18.5):

```

Remote fediverse servers
  ↓ HTTPS 443
Cloudflare edge: mastodon.300x3.com
  ↓ HTTP/2 tunnel (QUIC disabled after observed stream timeouts)
cloudflared-alwayson.service
  ↓ 127.0.0.1:3000
mastodon-web
mastodon-web + mastodon-sidekiq
  ↓ ao-egress-community
Remote ActivityPub/WebFinger endpoints

```

The storefront hostnames are not included in this tunnel ingress. Tunnel credentials remain in protected runtime storage and are never committed.

```

Superseded design (retained for history): workstation nginx
- listen 443 ssl; Let's Encrypt certificate (DNS-01)
- port 80 only as ACME/redirect listener
Not required with the tunnel path: edge TLS is provided by Cloudflare
and the origin stays loopback-only (Section 18.5).

```

TLS requirements:

- TLS is mandatory at the edge for all federation traffic (satisfied by Cloudflare edge termination for the tunnel-routed apex hostname).
- Tunnel credentials and origin certificate are stored 0400 under `~/.cloudflared/` and mirrored to KDE Wallet `ao-mastodon`; never in Git or this README. Revoke by deleting/re-creating the tunnel.
- `X-Forwarded-Proto: https` is supplied by cloudflared so Rails generates HTTPS URLs and Secure cookies (validated: instance JSON reports `streaming_api: wss://300x3.com`).

Outbound delivery path:

- Sidekiq delivers public activities to remote inboxes over HTTPS/443.
- Egress is restricted to `ao-egress-community` (Section 3) with HTTPS as the only approved protocol; no broad network membership.
- Rate and retry behavior are Mastodon defaults; no relay subscription is approved unless explicitly decided.

15.4.4 Federation Enablement Sequence

Status as of 2026-09-24:

1. **Done** — dedicated Cloudflare Tunnel `alwayson-mastodon-federation` and DNS route for `mastodon.300x3.com` created; storefront hostnames excluded.
2. **Done** — Cloudflare redirect rule narrowed to exclude `mastodon.300x3.com`; the static storefront redirect remains unchanged.
3. **Done** — Mastodon identity set to `LOCAL_DOMAIN=mastodon.300x3.com`; actor, WebFinger, and local actor documents verified.
4. **Done** — `ao-egress-community` attached to web/Sidekiq only; database, Redis, and streaming remain isolated.
5. **Done** — tunnel transport switched to HTTP/2 after QUIC stream timeouts; local and public health checks return HTTP 200.
6. **Done** — `@300x3@mastodon.social` resolved; public followers collection confirms both local accounts follow it.
7. **Done** — public post fetched; local mention records created and native notification processing repaired.
8. **Pending** — verify reverse follows using the remote following collection and local incoming relationship tables, then perform a fresh signed ActivityPub round-trip test.
9. **Pending** — bootstrap discovery: from Konqueror signed in at `https://mastodon.300x3.com`, follow at least one account on `mastodon.social`. Remote servers do not index this instance until first contact occurs. The storefront host `https://300x3.com` is a static site and is **not** routed to Mastodon.
10. **Pending** — validate public-post delivery to `mastodon.social` and reply/boost round-trips back to the local instance; then submit `300x3.com` to the `joinmastodon.org` directory (operator-approved).

15.4.5 Operational Boundaries After Enablement

- Only `public` visibility federates; `unlisted`, `private`, and `direct` do not appear on remote servers' explore pages. The OpenClaw draft-by-default and human-approval controls (Sections 11.2.2 and 15.2) remain mandatory for all public publication.
 - `post.sh` public-post guard remains the script-level approval gate.
 - Federated deletion is best-effort: remote servers may retain cached copies. Content published under this section must be treated as practically irreversible.
 - Publication audit logging (immutable, Section 15.2) must include the remote-delivery outcome for federated statuses.
-

16. Scripts and Operational Standards

16.1 Scripts Layout

```
/ALWAYSON/scripts/  
├── bootstrap/  
│   ├── 00-inventory.sh  
│   ├── 01-verify-photogrammetry-mount.sh  
│   ├── 02-install-host-dependencies.sh  
│   ├── 03-create-operational-layout.sh  
│   └── 04-create-podman-networks.sh  
├── deploy/  
│   ├── deploy-quadlet-domain.sh  
│   ├── validate-quadlet-domain.sh  
│   ├── enable-domain-services.sh  
│   └── rollback-domain.sh  
├── validation/  
│   ├── check-photogrammetry-mount.sh  
│   ├── check-open-ports.sh  
│   ├── check-network-isolation.sh  
│   ├── check-secrets-exposure.sh  
│   ├── check-gpu-runtime.sh  
│   ├── check-ledger-ingest.sh  
│   └── capture-version-matrix.sh  
├── mapping/  
├── radio/  
├── simulation/  
├── storefront/  
├── ledger/  
├── backup/  
├── restore/  
└── maintenance/
```

16.2 Script Standard

Every script begins with:

```
#!/usr/bin/env bash  
set -Eeuo pipefail  
IFS=$'\n\t'
```

Every script must:

- Use absolute paths.
- Validate prerequisites.
- Log in UTC.
- Avoid secrets.
- Support `--dry-run` for external or destructive activity.
- Use locks where concurrent invocation could corrupt data.
- Return meaningful exit codes.
- Avoid `eval`.
- Avoid unexamined `|| true`.
- Validate canonical paths before move or delete activity.
- Verify the photogrammetry mount before mapping activity.
- Refuse to delete outside explicitly approved and validated paths.
- Write an audit entry for operational changes.

17. Backup, Restore, Monitoring, and Completion Criteria

17.1 Backup and Restore Policy

Use a 3-2-1 strategy: three copies, two media types, and one off-host/off-site copy.

Frequency	Required activity
Continuous or 15-minute where enabled	Database WAL/archive strategy for critical recovery objectives
Hourly incremental	Configuration, manifests, sales records, field telemetry, current project data
Daily	PostgreSQL dumps for <code>salesdb</code> , <code>mastodon</code> , <code>webodm</code> , <code>grafana</code> , and <code>metabase</code> when active; Corda backup; mapping manifests; simulation exports; storefront releases
Weekly	Repository integrity check and off-host copy validation
Monthly	Isolated restore test
Quarterly	Full disaster-recovery exercise

Restore testing must:

1. Restore to an isolated test path or test host.
2. Validate database integrity.
3. Recalculate artifact hashes.
4. Compare hashes with stored manifests.
5. Verify associated Corda receipt/manifests where available.
6. Record operator, source backup ID, result, and exceptions.
7. Alert on failure.

17.2 Monitoring

Monitoring runs in `ao-admin` and is exposed only through VPN or authenticated administration access.

Monitor at minimum:

Component	Required metrics
Host	CPU, RAM, storage health, disk usage, temperature, GPU state, kernel errors
Podman/systemd	Unit state, restart loops, health, image digest
Mapping	Queue depth, failures, duration, disk space, CPU/GPU use
Field	Packet rate, RSSI, SNR, retries, replay rejections, spool depth, gateway uptime
Sales	Payment-verification failures, receipt failures, orders, API latency
AI/community	Model latency, request count, GPU use, approval queue, OAuth failures
Vehicle simulation	Scenario success, SITL/ROS/Gazebo health, result export
Fabrication simulation	Task state, collision/safety events, result export
Ledger	Corda health, ingest failures, certificate expiry, backup age
Backup	Last success, repository health, restore-test result, queue age

Alerts must cover disk pressure, backup failure, failed restore tests, container restart loops, unexpected listeners, failed payment verification, radio disconnection, WebODM backlog, GPU contention, expired certificates, and denied cross-domain traffic.

17.3 Completion Evidence

No installation or deployment agent may claim completion until it produces:

1. Host inventory report.
2. Photogrammetry-drive report with mount source, UUID, filesystem, free space, ownership, and permission validation.

3. Installed package and version matrix.
 4. Rootless and/or system Podman/Quadlet verification.
 5. GPU driver and container-runtime validation.
 6. Podman network list and domain-isolation results.
 7. IPv4 and IPv6 firewall/listening-port report.
 8. WebODM CPU-only smoke-test result using the dedicated drive.
 9. Vehicle-simulation smoke-test result.
 10. Fabrication-simulation smoke-test result.
 11. Heltec stable serial-device detection and LoRa-link test result.
 12. Ledger-ingestion test and Corda receipt result.
 13. Sales receipt-manifest test without payment secrets.
 14. Backup execution result.
 15. At least one isolated restore-test result.
 16. A current list of unresolved blockers, deviations, risks, and actions requiring human approval.
-

18. Approved Deviations and Open Decisions

| 18.1 Simulation Baseline Deviation

Decision: ROS 2 Lyrical and Gazebo Sim 10.5.0 are the installed baseline.

Status: Approved on 2026-08-24.

Rationale: The installed and smoke-tested environment differs from an earlier Jazzy/Harmonic draft.

Required control: Record versions, image digests, compatibility test results, and any future migration plan in the version matrix.

| 18.2 Corda Database Placement (CORRECTED)

Decision (ES.1, authoritative): Corda persistence uses **PostgreSQL**. PostgreSQL is the correct database for Corda.

Current implementation: `cordadb` is provisioned on the host PostgreSQL 18 cluster as a separate logical database with separate roles and backup scope. This is the correct placement and satisfies the decision above.

Root cause of the earlier deviation record: the Corda node was **set up with H2 instead of PostgreSQL**. That was the error. The H2 scaffold is the defect, not the PostgreSQL placement, and the earlier text in this section that described PostgreSQL as the deviation has been corrected accordingly.

Status: Corrected and closed as a placement question. Remaining work is the separate, still-open task of migrating the node off its H2 scaffold onto `cordadb` in PostgreSQL; until that is done the node is not production-ready and the operator key/certificate ceremony in section 18.3 still applies.

Required control: Document database roles, host-loopback binding, backup scope, restore procedure, and separation from sales/mapping databases.

| 18.3 Corda Deployment Blocker

Status: Blocked. MUST BE ADDRESSED BEFORE ADDITIONAL CORDA DEVELOPMENT

Condition: Corda node deployment requires the operator key and certificate ceremony.

Rule: Do not generate, replace, export, or activate production ledger keys without explicit operator approval and recorded ceremony output.

| 18.4 Payment Provider Decision

Status: Decided 2026-08-28.

Decision: PayPal (hosted checkout, provider-signed webhooks) plus **Zelle** for direct US payments PLUS COINBASE STABLECOIN (USDC), used from an operator-built custom HTML storefront. The storefront HTML will be developed externally (lovable.dev) and linked into this project; it remains static and is served from the pCloud Public Folder.

Controls required before enabling:

- PayPal: hosted checkout only; signature-verified webhook via `ao-ingress-payment` -> `ao-payment` verifier; credentials via §14.1 secret delivery; no PayPal secret material in the repo, logs, or pCloud.
- Zelle: manual reconciliation path only (equivalent to the wire-transfer policy in §7.2): operator-verified receipt, auditable reference record, explicit operator approval per §7.2. Zelle provides no public webhooks/API, so no automated verification is permitted until a documented control exists.
- Storefront: static HTML only; no server-side code in the pCloud Public Folder; all dynamic behavior goes through the payment and community adapters. Evaluate the lovable.dev-produced HTML against §4 data policy and the prohibited-paths list before linking.

The prior provider-evaluation draft is retained at `docs/compliance/payment-provider-evaluation.md` for record.

| 18.5 Mastodon Federation Edge and Identity Decision

Status: Decided and applied 2026-09-24 (closed). Remaining federation work is tracked in section 19.3.

Decision (operator):

- Serve Mastodon publicly at the dedicated hostname `mastodon.300x3.com` through Cloudflare Tunnel.
- Keep `300x3.com` and `www.300x3.com` on the existing static-site redirect; do not route the main website hostname to Mastodon.
- Edge transport is **HTTP/2** because QUIC stream timeouts were observed on this host. The tunnel service is `cloudflared-alwayson.service`.
- Mastodon identity is `LOCAL_DOMAIN=mastodon.300x3.com`; canonical accounts are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.
- `ao-egress-community` is attached only to `mastodon-web` and `mastodon-sidekiq`; database, Redis, and streaming remain on internal `ao-sales`.
- Origin ports remain loopback-only. TLS terminates at Cloudflare.

Verified state:

- Public actor and WebFinger endpoints return HTTP 200.
- The local web and health endpoints return HTTP 200.
- The remote account `@300x3@mastodon.social` lists both local accounts as followers, confirming local-to-remote follows. Its `following` collection is empty; reverse remote-to-local follows are not yet recorded.
- A public remote post was fetched and its local mention records were created; notification delivery was repaired through Mastodon's native notification service after the asynchronous worker failed to materialize the rows.

Resolution condition: verify reverse follows using the remote account's `following` collection and the local incoming relationship tables. Do not mark the reverse direction complete based only on a local outgoing request.

19. Open Implementation Items

Everything in this document is either a standard (stated in sections 1–18, above) or work that remains to be done. There is no third category. **Standards are not repeated here**, even where a standard has an unmet acceptance test — the unmet test belongs in the table below, the rule itself belongs in its own section.

This is the single list of remaining implementation work. Each row names the standard it serves, so the requirement is never lost. Completed and verified items are not listed; they are recorded as evidence in section 20.

19.1 Blocking — the system is not production-ready without these

#	Item	Standard served	Acceptance criteria	Blocks
1	Corda key/certificate ceremony	§18.3, §11	Operator ceremony performed and output recorded. No production ledger keys generated, replaced, exported, or activated without explicit operator approval.	Ledger core, all §11 flows
2	Corda H2 → PostgreSQL migration	ES.1, §18.2	Node moved off its H2 scaffold onto <code>cordadb</code> in PostgreSQL 18; correlation join by receipt number, serial number, and UTC timestamp proven.	Ledger core
3	Controlled ingress/egress adapters	§5.2	<code>ao-ingress-payment</code> , <code>ao-egress-archive</code> , <code>ao-egress-community</code> , <code>ao-build-update</code> implemented with destination allowlists, validated TLS, separate credentials, and connection logging.	External payment, archive, community connectivity
4	Unattended secret delivery decision	§14.1, §18	Either migrate <code>mastodon-db</code> , <code>sales-db</code> , and <code>webodm-db</code> to Podman secrets or systemd credentials, or record an approved deviation with compensating controls, before any production declaration.	Production declaration
5	Mapping runtime designation	§13.2	WebODM runtime finally designated rootless, system-level, or mixed, and the mixed-store deviation in §13.2 closed or confirmed.	Mapping production declaration

19.2 Payments, sales, and ledger

| # | Item | Standard served | Acceptance criteria |

19.3 Community, federation, and local AI

#	Item	Standard served	Acceptance criteria
12	Mastodon configuration drift reconciliation	§15.4	<code>config/mastodon/instance-policy.yaml</code> , <code>mastodon.env.example</code> , <code>version-matrix.yaml</code> , <code>secrets/mastodon/mastodon.env</code> , and <code>fetch-mastodon-env.sh</code> all reconciled to <code>mastodon.300x3.com</code> . Do this before the next Mastodon restart — the helper emits the superseded apex value unconditionally.
13	Reverse-follow validation	§15.4.4	Confirmed from the remote <code>following</code> collection and local incoming relationship tables, never inferred from local outgoing state.
14	Fresh signed ActivityPub round trip	§15.4.4	Run after the notification-worker fix; reply/boost round trip received locally.
15	Remote account approval/rejection record	§15.4.5	Recorded separately from local account follow state.
16	OpenClaw OAuth and conversation validation	§15.2	OAuth completes over HTTPS at the federation origin; OpenClaw posts a threaded reply per mention; bridge posts only to the local instance.
17	Mastodon service-account consolidation	§14.1.1, §20.0	<code>always-on-sales</code> (UID 993) placement folded back to the operator account. No separate service-account user is intended.
18	<code>300x3.com</code> email routing / MX	§15.3	Delivery confirmed or formally deferred.
19	Per-modal purchase buttons, HTML-300X3	§7.1.1	Implemented in the repo and the static export mirrored to the pCloud Public Folder.

19.4 Field, radio, and simulation

#	Item	Standard served	Acceptance criteria
20	RF characterization on both bands	\$9.5	RSSI, SNR, noise floor, packet loss, retry behaviour, and airtime recorded on both RNodes. Closure is a recorded finding, not a fix — if the interference is benign ambient noise, record that. No corrective action unless measurement shows a real fault.
21	End-to-end field link test	\$9.2, \$9.5	Unicast and broadcast proven over each RF path; fail-safe verified on radio, serial-path, and peer loss; no live flight-control path enabled during testing.
22	Cross-band isolation	\$9.5	915 MHz and 917 MHz isolation measured; interference classified as in-band, adjacent-band, harmonic, or spurious.
## 19.5 Operations, reporting, and documentation			

#	Item	Standard served	Acceptance criteria
29	Metabase persistence and first read-only query	\$15.1, \$17.2	State survives restart; a protected read-only reporting query succeeds; no source writes. Low priority.
30	Version matrix refresh	\$14.2	\$14.2 currently lags \$20 on several rows. Refresh from the host; the matrix and the evidence table must agree.
31	Version-matrix capture automation	\$14.2, \$16	<code>scripts/validation/capture-version-matrix.sh</code> documented as the producer, with a stated refresh requirement.
32	<code>apparmor-utils</code> and GPU toolkit packages	\$12.3	Install list omits packages that later verification blocks assume exist (<code>aa-status</code> check, CDI/GPU access). Reconcile the install list with the verification steps.
33	Asserting install verification	\$12.3	The \$12.3 verify block prints values without asserting them, and the cgroup check is silent on failure. Add real assertions.
34	Ledger socket-bridge diagnosis	\$17.1	<code>scripts/validation/check-ledger-ingest.sh</code> resolved, or the pending operator-run privileged command executed.
35	Scripts layout completeness	\$16.1	Layout is missing the <code>sales/</code> directory and <code>validate-sale-receipt.sh</code> , both referenced elsewhere. Add or repoint them.
36	Restore-test script contract	\$17.1	The seven-step restore test is unowned; state that the <code>check-*.sh</code> scripts implement it, or the requirement has no executor.
37	WebODM folder validation	\$8.5	Tree, ownership, sentinel, and checks validated; WebODM starts only with required validated storage.
38	GPU scheduling and admission policy	ES.1	LM Studio, SketchUp, Gazebo, and WebODM batch scheduling matches the documented priority order.

| 23 | Reticulum gateway listener review | \$9.4, \$9.5 | [0.0.0.0:4242](#) reviewed against field-domain firewall policy; reachability decided rather than left unverified. | | 24 | Interface status report | \$9.3 | Timestamped connected/disabled/error report for all 32 configured interfaces. | | 25 | `umsgpack` persistence error | \$9.3 | Classified, or formally accepted as a historical bounded-ratchet defect with restart-persistence evidence. | | 26 | Gazebo GUI clients and DDS policy | \$10.1, \$10.2 | Vehicle and fabrication GUI clients deployed; separate DDS/interface policy decided. | | 27 | `/ALWAYSON` Gazebo subfolder | \$10.2 | Path confirmed by the operator. Currently recorded as an open decision, not a guess. | | 28 | QGroundControl interactive workflow | \$10.1 | Interactive SITL workflow validated end to end. |

|---|---|---|---| | 6 | Payment credentials into KDE Wallet `ao-payment` | \$14.1 | Folder provisioned per \$14.1.1; entry stored through `kwallet-provision.sh`; no secret in Git, logs, HTML, or Corda. | | 7 | Payment verifier and normalized event model | \$7.2, \$7.3 | A test payment event produces a verified normalized record. | | 8 | Sales API and receipt/fulfillment workflow | \$7.3, \$15.1 | A sales receipt manifest can be generated without exposing sensitive data. | | 9 | `salesdb` schema initialization | \$3.3.1, \$15.1 | Live application schema initialized; read-only reporting views defined. | | 10 | Corda ingest accepts only approved signed data | \$4.4, \$11.2 | Ledger-ingest receives signed, minimized manifests only, with authorization, idempotency, replay defence, and audit. | | 11 | pCloud archive credentials | \$11.6, \$17.1 | Credentials provisioned into `ao-archive`; non-destructive encrypted replication test approved and run.

Presence-only checks — never print, copy, or export values. |

20. Current Verification Evidence

Status as of 2026-09-25. Each row records the outcome of a check against the running system, not design intent. Where a component is misleading in the operator surface, the discrepancy is stated.

Item	Evidence	Status
Host inventory	Inventory report completed	Complete
Photogrammetry drive	UUID verified; directory tree created	Complete
Package/version matrix	Captured and refreshed	Complete
GUI boundary matrix (section 19)	<code>config/platform/gui-boundary-matrix.yaml</code> created; 10 entries validated (YAML), covering all Section 6.A scope items	Partial
Rootless Podman and Quadlet	Verified; mixed-store deviation documented	Complete with deviation
GPU runtime	Driver/CDI verified; CPU baseline and GPU smoke completed	Complete
Domain network isolation	Internal workload networks and test verified	Complete
Firewall and ports	UFW active; prior <code>:80</code> and <code>:1716</code> exposure cleared	Complete
WebODM smoke test	<code>apt-76</code> ; 76 images; GPU-enabled orthophoto produced	Complete
Vehicle simulation	Headless Gazebo 300-iteration and ROS-Gazebo bridge test	Complete
Fabrication simulation	Headless Gazebo 300-iteration and bridge test	Complete
Heltec/LoRa detection	Heltec V3 connected; stable by-id + <code>/dev/heltec-v3</code> path, udev rule installed, <code>detect-heltec.sh</code> OK, serial probe received c0-framed packets 2026-08-31; LoRa-link test pending ao-field gateway	Partial
Corda receipt	Corda 5.2.2 scaffolded; key ceremony pending	Blocked
Sales receipt manifest	Sales DB deployed; provider/API pending	Partial
Backup	Encrypted restic snapshot <code>548d9910</code> completed; recurring schedule automated 2026-08-31 (restic nightly 03:30 timer, weekly integrity verify Sun 04:30, nightly domain DB dumps 03:00 for mastodon/sales/webodm); verification snapshot <code>32be2a1c</code> saved	Complete
Restore	File hash validated; database 14/14 tables restored	Complete
Monitoring stack (ao-admin)	Prometheus + node_exporter + Grafana run as <code>scottw</code> Quadlet units on <code>ao-admin</code> . Grafana application state is genuinely PostgreSQL-backed against the host cluster over the <code>/var/run/postgresql</code> socket (<code>/api/health</code> reports <code>database: ok</code>), and Prometheus is its only registered datasource. Both Prometheus targets scrape <code>up</code>	Complete
Metabase reporting (ao-admin)	Working. Metabase runs on the host and serves its login page in the browser, which is the expected operator surface. Operator-confirmed 2026-09-28; this supersedes the earlier "not serving" finding. The earlier record described a containerised <code>ao-metabase</code> instance failing during application-database setup and cycling under <code>Restart=on-failure</code> ; that container and that fault are not the service the operator uses	Complete — login page serves in the browser
Mastodon local stack (ao-sales)	All 5 containers run under the <code>always-on-sales</code> service account in a separate rootless store and systemd user manager (Section 20.0); <code>ao-mastodon-web</code> / <code>ao-mastodon-streaming</code> are masked in the <code>scottw</code> manager as a duplicate guard. Web <code>127.0.0.1:3000</code> and streaming <code>127.0.0.1:4000</code> verified; <code>/api/v1/instance</code> reports <code>mastodon.300x3.com</code> v4.3.7	Complete (live, federated)
Mastodon federation edge	Dedicated Cloudflare Tunnel <code>always-on-mastodon-federation</code> for <code>mastodon.300x3.com</code> ; HTTP/2 connector active; actor and WebFinger 200; storefront hostnames preserved; <code>LOCAL_DOMAIN=mastodon.300x3.com</code> ; canonical accounts <code>admin@mastodon.300x3.com</code> and <code>bot@mastodon.300x3.com</code> ; <code>ao-egress-community</code> attached to web/Sideiq only; local-to-remote follows confirmed; reverse-follow validation pending	Partial — signed round-trip and reverse-follow evidence remain
WebODM operator workflow restart	Stack is rootless (<code>scottw/mapping</code> store); system-store recovery step correctly found no system-store containers — no action needed	Complete

Item	Evidence	Status
ArduPilot SITL MAVLink	ao-ardupilot-sitl.service flags fixed; HEARTBEAT (sysid 1, QUADROTOR, ArduPilot) validated over tcp:127.0.0.1:5760 via pymavlink	Complete
Heltec firmware	RNode firmware 1.85 recorded via rnodeconf; EEPROM valid; signature unverified (operator signing option)	Partial
Reticulum executable	Standalone RNS 1.4.2 available at <code>/home/scottw/.local/bin/rnsd</code> ; active Reticulum runtime is embedded in MeshChatX	Complete with embedded version pending
MeshChatX deployment	Native headless backend running since 2026-09-24 11:02 local time; local UI bound to <code>127.0.0.1:18000</code> ; desktop metadata declares 4.9.1	Complete with running-version verification pending
Reticulum interface configuration	29 TCP clients use <code>interface_enabled = true</code> ; two RNodes use <code>interface_enabled = true</code> ; one Backbone uses <code>enabled = yes</code> ; zero explicitly disabled	Complete
Reticulum runtime participation	Logs show auto-connections, peering, announces, and LXMf/Nomad network announcements	Partial — per-interface health not yet captured
Reticulum connectivity	Startup logs contain timeouts, network-unreachable errors, connection refusals, and reconnect cycles for named and discovered interfaces	Partial
Reticulum public gateway	MeshChatX is bound to <code>0.0.0.0:4242</code> ; the host had <code>192.168.87.135/24</code> on Wi-Fi	Partial — binding verified; firewall and packet reachability not verified
Two-radio Reticulum initialization	Both serial paths exist and MeshChatX logged both RNodes as configured and powered up on 2026-09-24	Complete
RNode band feedback	Functional feedback observed on both 915 MHz and 917 MHz paths	Issue — source and severity uncharacterized
MeshChatX cryptographic-state persistence	12,364 historical <code>umsgpack</code> errors; error block ends before newer 16:29Z and 16:51Z startup entries	Issue — controlled-restart test required
MeshChatX version provenance	Desktop metadata declares 4.9.1; executable hash matches the local manifest; running version remains unverified; repository cache contains a 4.8.4 wheel	Partial

21. Status References

Review the following before changing the platform:

```
README.md
VERSION
git log
docs/compliance/installation-status.md
/ALWAYSON/
```

The current repository README, local working folder, verification evidence, version matrix, and issue log must be reviewed before beginning new work.

Current implementation references:

```
/home/scottw/.openclaw/openclaw.json
quadlet/sales/ao-egress-community.network
quadlet/sales/ao-mastodon-web.container
quadlet/sales/ao-mastodon-sidekiq.container
scripts/mastodon/federate-local.sh
/home/scottw/.cloudflared/config.yml
```

Do not add API keys, passwords, tunnel credential JSON, or other secrets to this reference list.

Appendix A. Revision History

Revision	Date	Change
v1	2026-09-28	Consolidated single-file plan.
v2	2026-09-28	Structural and consistency revision. Removed the duplicated title/banner block. Renamed the summary sections ES.1-ES.4 to end the collision with body sections 1-21. Resolved Tokodon references against ES.1. Qualified Grafana as optional. Scoped IPFS to post-sale. Deleted the duplicated prohibited-secret list in 11.4. Numbered the unnumbered headings in 11.2 and repaired the broken ordered lists and the truncated 15.4 status prose. Separated two confusable federation/local-operation identifiers.
v4	2026-09-28	Operator decisions applied, topology consolidated, layout optimized.

What v4 changed

Operator decisions recorded (2026-09-28). These are the operator's calls and now govern the document:

- Login-gated secret delivery is intended.** Services that consume KDE Wallet secrets start after Plasma login and are not expected to start unattended before a password is entered. The ~60s wait is the bounded startup allowance, not a fallback that must survive a passwordless boot.
- No separate service account.** All services run under the operator account. The `alwayson-sales` (UID 993) Mastodon placement in section 20.0 is legacy, not intended, and should be consolidated back to the operator account.
- KDE Wallet remains the secret authority.** The `org.kde.kwalletd6` bus name and its method names are flagged in section 14.1.1 for verification on the host. The authority is not in question; only the access details are unconfirmed.
- Corda uses PostgreSQL.** PostgreSQL is the correct database for Corda. The node was scaffolded on **H2** and that H2 scaffold is the defect. The `cordadb` placement on the host PostgreSQL 18 cluster is correct and section 18.2 is corrected accordingly.
- Metabase works.** It runs on the host and serves its login page in the browser. The earlier "not serving" finding described a containerised `ao-metabase` instance that is not the deployed service path.
- RF interference may simply be interference.** WORK 000700 now closes on a recorded characterization, not a fix. No corrective action is required unless measurement shows a real fault.

Consolidated topology. ES.2 is now a single authoritative master diagram covering the entire project — external entry points, the four adapters, all ten `ao-*` workload networks, the ledger path, post-sale IPFS transfer, secret delivery, and monitoring — with a reading key answering the seven questions a reader actually has. §3.1 is relabelled a detail view of it so the two cannot drift apart.

Work and issue items consolidated. The former section 19 was a 588-line WORK 000005-style prose log. Per the operator's rule that work items must not be labelled as work items in the body, it is now **19. Open Implementation Items**: one table of 38 remaining items, each naming the **standard it serves** so the rule and its unmet test are never confused. Anything that is a standard now lives only in its own section (1-18); completed and verified items live only in section 20 evidence. The WORK 0000xx and ISSUE 000xxx identifiers, and their in-text cross-references, are gone.

Simulation renders relocated. Three Gazebo model views that were referenced at the end of the report are placed in section 10.2 with captions, where the fabrication and facility content they illustrate actually lives.

Layout optimization. The 6.A.1 GUI/network table repeated the same ten-network "no access" list in all 17 rows, making every row roughly 800-900 characters wide. The rule is now stated once above the table and the per-row column carries only the owning network, cutting the widest row from 927 to about 200 characters while preserving identical meaning.

Open items

Item	Status	Owner action
Corda H2 → PostgreSQL migration	Open, blocking	Migrate the node off its H2 scaffold onto <code>cordadb</code> ; then run the operator key/certificate ceremony (§18.3).
kwalletd6 D-Bus access details	Open, low	Confirm the bus name and method names on the running host, then record the verified values in §14.1.1. Wallet authority is settled.
Mastodon service account consolidation	Open, low	Fold the <code>alwayson-sales</code> (UID 993) placement back to the operator account (§20.0).

Item	Status	Owner action
Metabase persistence + first read-only query	Open, low	Confirm state survives restart and the first protected read-only reporting query succeeds with no source writes (WORK 000601).
RF characterization (WORK 000700)	Open, observational	Measure and record. Closes on a finding, benign or not; no fix implied.
/ALWAYSON Gazebo subfolder	Open, trivial	Confirm the path (§10.2).
Version matrix vs evidence table	Open, low	§14.2 lags §20 on several rows; refresh it from the host.